Static Timing Analysis Lab

(BECVL0552)



BACHELOR OF TECHNOLOGY

Degree

In

Electronics Engineering (VLSI Design & Technology)

ADITYA RAJ

Roll No. - 2401331730005

Submitted to

Mr. Ankur Bharti

Mr. Sagar Roy

NOIDA INSTITUTE OF ENGINEERING & TECHNOLOGY,

GREATER NOIDA

INDEX

S.NO.	TITLE OF PROJECT	PAGE NO.	SIGNATURE
1	Introduction to Synopsys PrimeTime/Cadence Tempus Flow: Loading libraries, netlists, and running basic STA reports	1-3	
2	To Simulate Clock and I/O Constraint Definition using create_clock, set_input_delay, and set_output_delay.	4-8	
3	To Simulate Setup Time Analysis and Critical Path Identification	9-16	
4	To perform Hold Time Analysis and fix Hold Time Violation using Buffer Insertion in Vivado.	17-21	
5	To simulate and analyze the effect of clock skew and clock uncertainty on the timing performance of a synchronous digital circuit using Xilinx Vivado.	22-25	
6	To Simulate the timing Exception Analysis using set_false_path.	26-30	
7	To simulate Multicycle Path Timing Analysis using the set_multicycle_path constraint in Xilinx Vivado.	31-34	
8	To study clock-gating timing verification and enable-signal behaviour using Verilog HDL and Xilinx Vivado.	35-37	
9	To determine and simulate multi-Mode Multi-Corner (MMMC) Timing Analysis across PVT Corners.	38-41	
10	To determine On-Chip Variation (OCV/AOCV/SOCV) and Timing Derating Analysis.	42-45	
11	To analyse Clock Domain Crossing (CDC) Analysis using Synchronizer Circuits.	46-48	

EXPERIMENT NO. 1

AIM: Introduction to Synopsys PrimeTime/Cadence Tempus Flow: Loading libraries, netlists, and running basic STA reports

TOOLS REQUIRED: PC, PrimeTime/Cadance Tempus Flow/Vivado

INTRODUCTION TO TOOL:

Vivado is a design and analysis tool used for developing and verifying digital designs. It provides facilities for synthesis, implementation, and **Static Timing Analysis (STA)**. In STA, timing constraints such as clock period, input delay, and output delay are applied to the design. Vivado analyzes timing paths and generates reports for **setup time, hold time, slack, and critical paths**. These reports help identify timing violations and evaluate the timing performance of the design.

PROCEDURE:

STEP 1: Launch Vivado and create a project

Open Vivado and choose **Create Project**. Enter the project name and location. Select **RTL Project**, then choose the device or board specified by the laboratory. Set the target language to Verilog as required.

STEP 2: Add and edit a design source.

Choose **Add Sources → Add or Create Design Sources**. Create a Verilog file such as and_gate.v and enter the required HDL code. Save the file and verify the design hierarchy in the **Sources** window.

STEP 3: Check the RTL schematic

Set the intended module as the design top. Under **RTL Analysis**, open **Elaborated Design → Schematic**. Verify that the ports and logic correctly represent the HDL description.

STEP 4: Prepare a testbench

Choose **Add Sources → Add or Create Simulation Sources** and create a testbench. Instantiate the design and apply suitable input combinations. Select the testbench as the simulation top.

STEP 5: Run behavioural simulation

Select **Run Simulation → Run Behavioral Simulation**. Observe the input and output waveforms and verify that the simulated output matches the expected functionality.

STEP 6: Run Synthesis

Select **Run Synthesis** from the Flow Navigator. After synthesis completes, open the synthesized design and check for synthesis errors or warnings. Synthesis converts the RTL description into a hardware netlist.

STEP 7: Add Timing Constraints

Create or add an **XDC constraint file**. Define the required clock using create_clock and specify input/output timing constraints such as set_input_delay and set_output_delay when required. These constraints define the timing requirements for STA.

STEP 8: Run Implementation

Select **Run Implementation** from the Flow Navigator. Implementation performs placement and routing of the synthesized design. After completion, review any implementation errors and warnings.

STEP 9: Perform Static Timing Analysis

After implementation, open **Implemented Design → Timing Analysis** or select **Report Timing Summary**. Vivado analyzes the timing paths using the applied constraints and reports parameters such as **slack, setup, hold, and critical paths**.

STEP 10: Analyze Timing Reports

Open the **Timing Summary** and examine **Worst Negative Slack (WNS)**, **Total Negative Slack (TNS)**, setup violations, and hold violations. Identify the critical timing paths and determine whether the design satisfies the required timing constraints.

STEP 11: Fix Timing Violations

If timing violations are present, modify the RTL, constraints, or design implementation as appropriate. Re-run synthesis and implementation and perform STA again to verify that the timing requirements are satisfied.

STEP 12: Generate Final STA Report

Record the final **timing summary, critical path, setup/hold slack, WNS, and TNS** values. A design is considered timing-clean when the required timing constraints are met without setup or hold violations.

BASIC EXAMPLE

```
module and_gate (
   input wire a,
   input wire b,
   output wire y

);
   assign y = a & b;

endmodule
```

module and endmodule mark the circuit boundary. Inputs a and b are one-bit signals; output y is driven continuously by assign. The & operator performs bitwise AND, so y is 1 only when both inputs are 1. This is an introductory example, not a screenshot-derived simulation result.

Important Vivado Terms for STA Lab

1. **Project** – Workspace containing RTL, constraints, simulation, and design files.
2. **RTL** – Register Transfer Level description of the digital circuit.
3. **Elaboration** – Checks and builds the RTL design hierarchy before synthesis.
4. **Synthesis** – Converts RTL code into a gate-level/netlist representation.
5. **XDC** – Constraint file used to define timing and physical constraints.
6. **create_clock** – Defines the clock and its period.
7. **Implementation** – Performs placement and routing of the synthesized design.
8. **STA** – Analyzes timing paths without applying input test vectors.
9. **Setup Time** – Minimum time data must be stable before the clock edge.
10. **Hold Time** – Minimum time data must remain stable after the clock edge.
11. **Slack** – Difference between required time and actual arrival time.
12. **WNS** – Worst Negative Slack; indicates the worst timing violation.
13. **TNS** – Total Negative Slack; sum of negative slack across violating paths.
14. **Critical Path** – Timing path with the worst slack.
15. **Timing Summary** – Report containing setup, hold, WNS, TNS, and timing status.
16. **Timing Constraint** – Requirement that specifies when signals must arrive.

References

1. AMD Xilinx Vivado Design Suite User Guide – Design Flows Overview (UG888)
2. AMD Xilinx Vivado Design Suite User Guide – Using Constraints (UG903)
3. AMD Xilinx Vivado Design Suite User Guide – Design Analysis and Closure Techniques (UG906)
4. AMD Xilinx Vivado Design Suite User Guide – Synthesis (UG901)
5. AMD Xilinx Vivado Design Suite User Guide – Logic Simulation (UG900)

These references cover Vivado project creation, RTL design, synthesis, XDC timing constraints, implementation, and Static Timing Analysis (STA).

EXPERIMENT NO. 2

AIM: To Simulate Clock and I/O Constraint Definition using create_clock, set_input_delay, and set_output_delay.

TOOLS REQUIRED: PC, PrimeTime/Vivado

INTRODUCTION:

Clock and I/O constraints are essential for performing Static Timing Analysis (STA) of a digital design. The **create_clock** command is used to define the clock signal and its timing characteristics, such as the clock period and waveform. The **set_input_delay** command specifies the delay of external input signals with respect to the clock, while **set_output_delay** defines the required timing for signals at the output ports. These constraints allow Vivado to accurately analyze the timing of input-to-register, register-to-output, and register-to-register paths. Proper constraint definition is necessary to identify timing violations and ensure that the design meets its required timing specifications.

1. Create a Launch Flip-Flop (FF1) that captures input a on every positive edge of the clock.
2. Implement a combinational logic block between the two flip-flops using the following Boolean expression: comb_out = (q1 & b) | (~q1)
3. Create a Capture Flip-Flop (FF2) that captures the output of the combinational logic on the positive edge of the same clock.
4. Both flip-flops must support an active-high asynchronous reset. Use a single clock signal for both flip-flops.

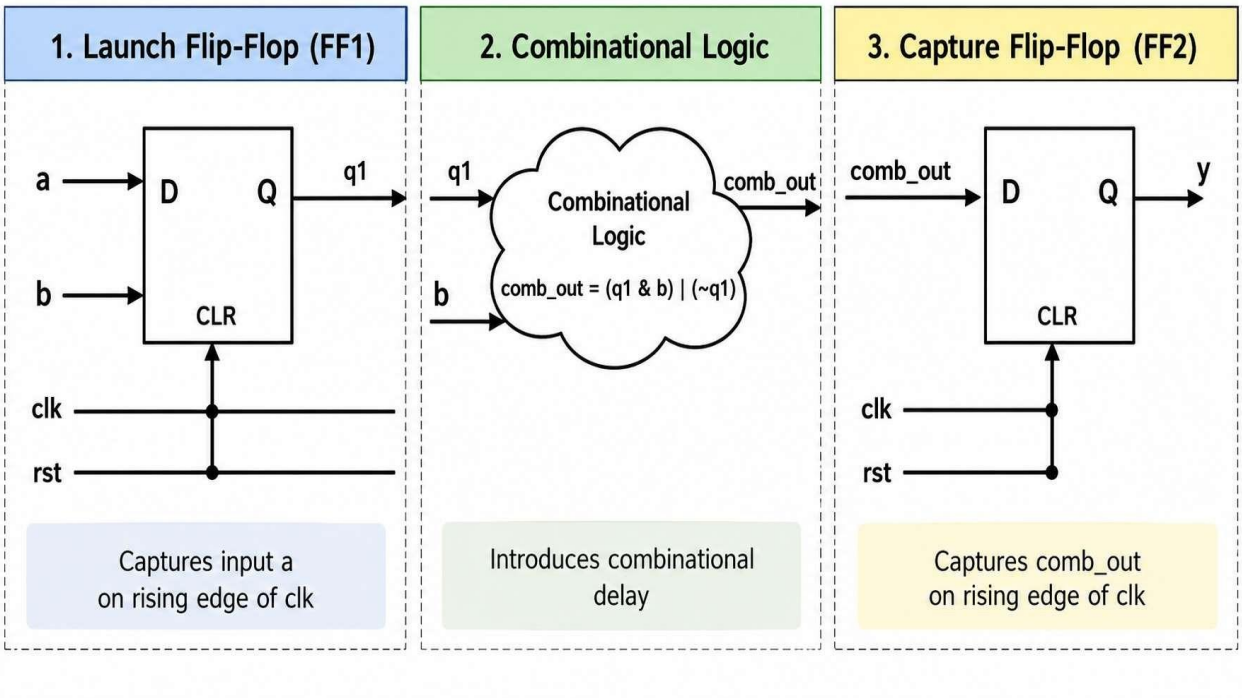


Figure 2.1: *Launch and Capture flip flop*

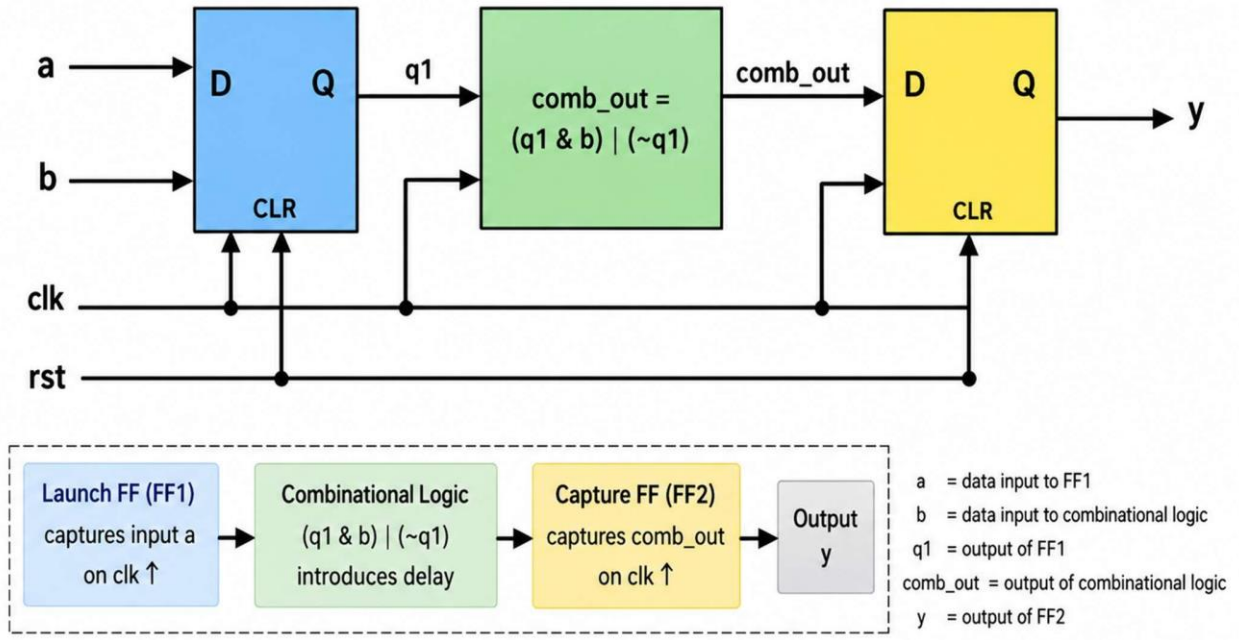


Figure 2.2: *sequential circuit containing two positive-edge triggered D Flip-Flops with combinational logic between them.*

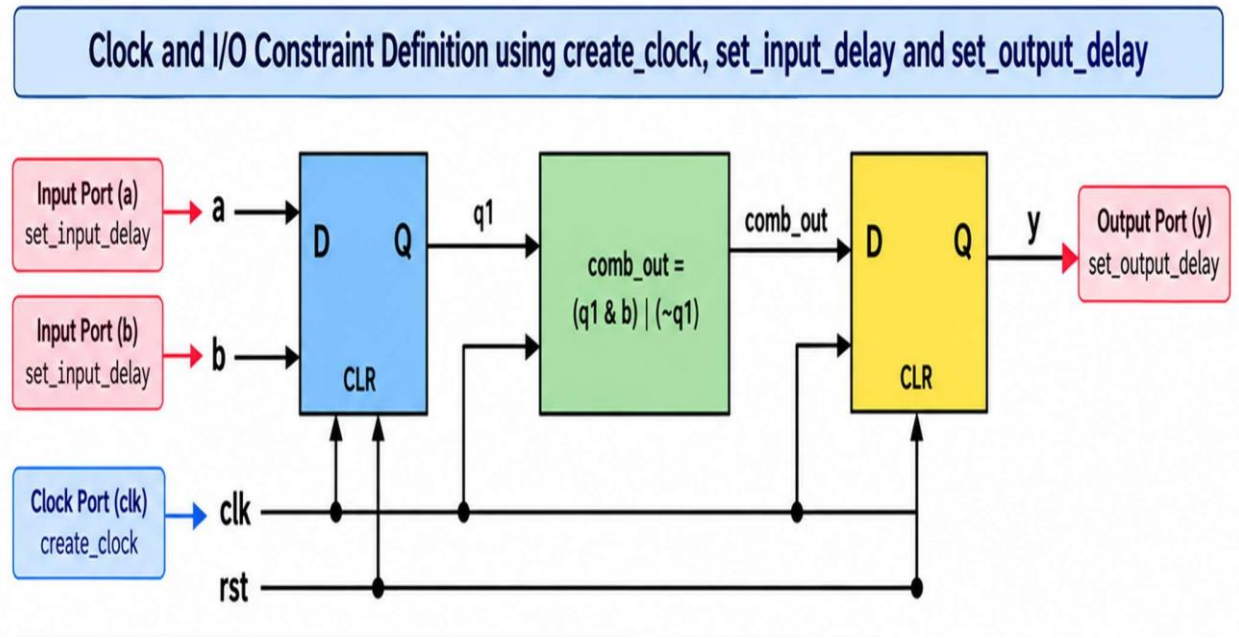


Figure 2.3: Clock and I/O Constraint

Example XDC Constraints

```
# 1. Define the clock (clk period = 10 ns, 100 MHz)
create_clock -name clk -period 10.0 [get_ports clk]

# 2. Define input delays (relative to clk)
set_input_delay -clock clk -max 2.0 [get_ports a]
set_input_delay -clock clk -min 0.5 [get_ports a]

set_input_delay -clock clk -max 2.0 [get_ports b]
set_input_delay -clock clk -min 0.5 [get_ports b]

# 3. Define output delays (relative to clk)
set_output_delay -clock clk -max 2.0 [get_ports y]
set_output_delay -clock clk -min 0.5 [get_ports y]
```

Figure 2.4: *XDC Constraints*

PROCEDURE:

1. Open the Vivado project containing the sequential circuit and ensure that the required Verilog design is set as the top module.
2. Create an XDC constraint file by selecting Add Sources → Add or Create Constraints → Create File.
3. Define the clock constraint using the create_clock command. Specify the clock port and required clock period.
4. Define the input delay using set_input_delay to specify the delay of external input signals relative to the defined clock.
5. Define the output delay using set_output_delay to specify the required timing relationship of output signals with respect to the clock.
6. Run Synthesis and then Run Implementation so that Vivado can analyze the constrained timing paths.
7. Open Timing Summary/Report Timing Summary and check the setup time, hold time, slack, WNS, TNS, and critical paths.
8. Verify that the applied clock and I/O constraints are correctly recognized and that the design satisfies the required timing constraints
9.

VERILOG CODE

Typed from the supplied screenshot; declarations and spacing are consolidated without changing behaviour.

```
module setup(
```

```
    input wire clk, rst, a, b,
    output reg y
);
    reg q1;
    wire comb_out;
    // First flip-flop: active-high asynchronous reset
    always @(posedge clk or posedge rst) begin
        if (rst) q1 <= 1'b0;
        else     q1 <= a;
    end
    assign comb_out = (q1 & b) | (~q1);
    // Second flip-flop
    always @(posedge clk or posedge rst) begin
        if (rst) y <= 1'b0;
        else     y <= comb_out;
    end
endmodule
```

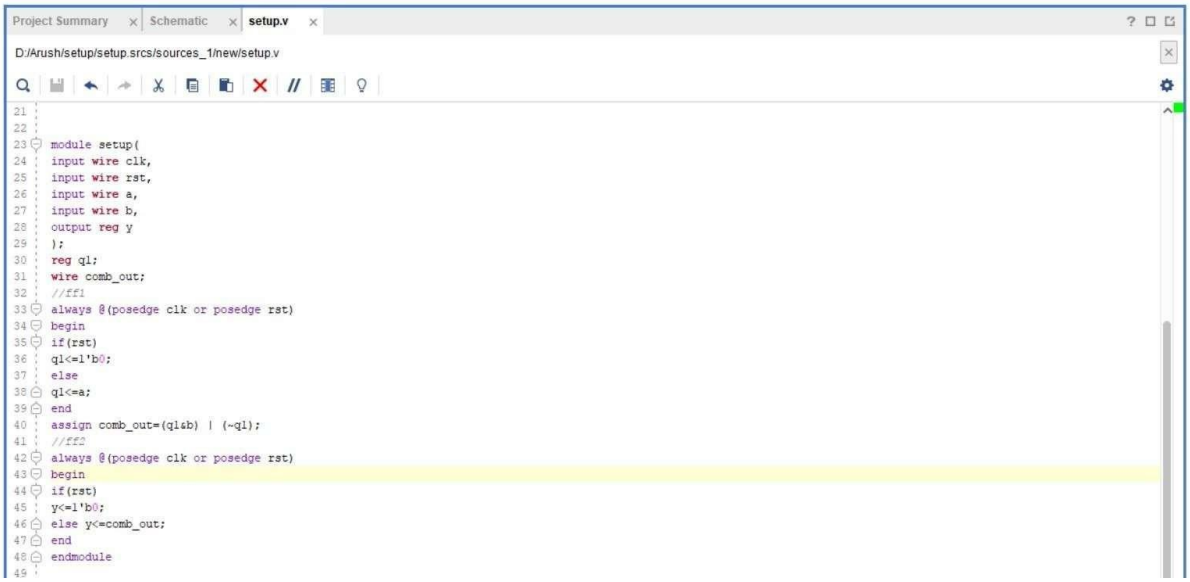


Figure 2.5: *Supplied Verilog source for the setup module.*

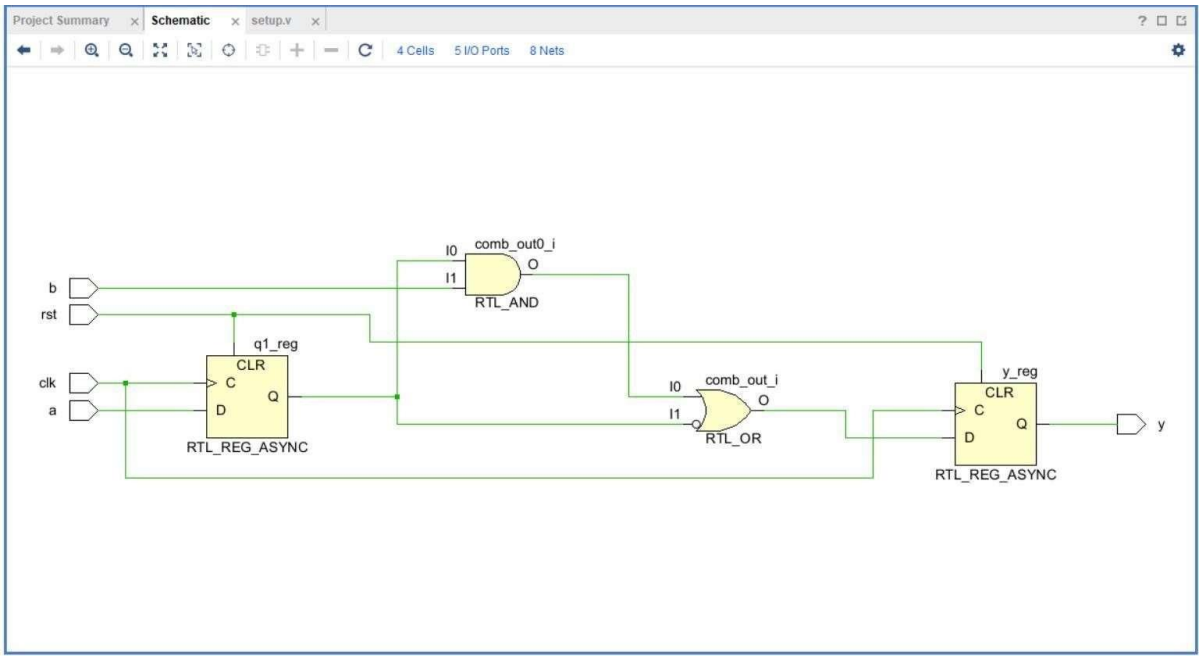


Figure 2.6: *Supplied RTL schematic showing two registers and AND/OR logic.*

OBSERVATIONS:

Clock edge	**q1 after edge**	**y after edge**	**Reason**
15 ns	0	1	Old q1 = 0
25 ns	1	1	y still uses old q1 = 0
35 ns	1	0	Old q1 = 1 and b = 0
45 ns	1	1	b = 1
65 ns	0	0	Old q1 = 1 and b = 0
75 ns	0	1	Old q1 = 0

EXPERIMENT NO. 3

AIM: To Simulate Setup Time Analysis and Critical Path Identification

TOOLS REQUIRED: PC, Vivado Design Suite, Verilog design source setup_analysis.v and simulation source tb_setup_analysis.v.

THEORY:

Inputs A, B, C and D are four-bit buses. Four input registers store these values at a rising clock edge. The adder combines A_reg and B_reg; the XOR stage combines the sum with C_reg; the AND stage masks that result with D_reg. The output register stores the final result in Y.

The datapath implements Y_next = ((A_reg + B_reg) XOR C_reg) AND D_reg. The addition is stored in a four-bit wire, so a carry beyond bit 3 is discarded: the stored sum is modulo 16. XOR produces 1 when corresponding bits differ. AND keeps a result bit only where the corresponding D_reg bit is 1.

CLOCK AND RESET:

The reset is synchronous and active high. Unlike Experiment 2, raising rst does not immediately clear the registers; reset is applied at the next rising clock edge. With reset inactive, the input registers capture new data and Y captures the result of the previous registered data. The delay from input capture to the matching output is one clock period.

SIGNAL DEFINITIONS:

Signal	Width	Purpose
clk / rst	1 bit each	Clock and synchronous reset
A, B, C, D	4 bits each	External input buses
A_reg ... D_reg	4 bits each	Stored inputs
add_out	4 bits	Truncated sum
xor_out / and_out	4 bits each	Intermediate logic results
Y	4 bits	Registered output

PROCEDURE:

Add setup_analysis.v to a Vivado RTL project and set it as the design top. Inspect the elaborated schematic. Add tb_setup_analysis.v to simulation sources, set it as the simulation top and run the testbench for 80 ns. Observe clk, rst, A, B, C, D and Y, adding internal signals when needed.

SCOPE OF ANALYSIS:

The supplied files describe a registered datapath. No timing constraints or timing report were supplied, so physical setup-time closure or maximum clock frequency cannot be concluded from the code or RTL image alone.

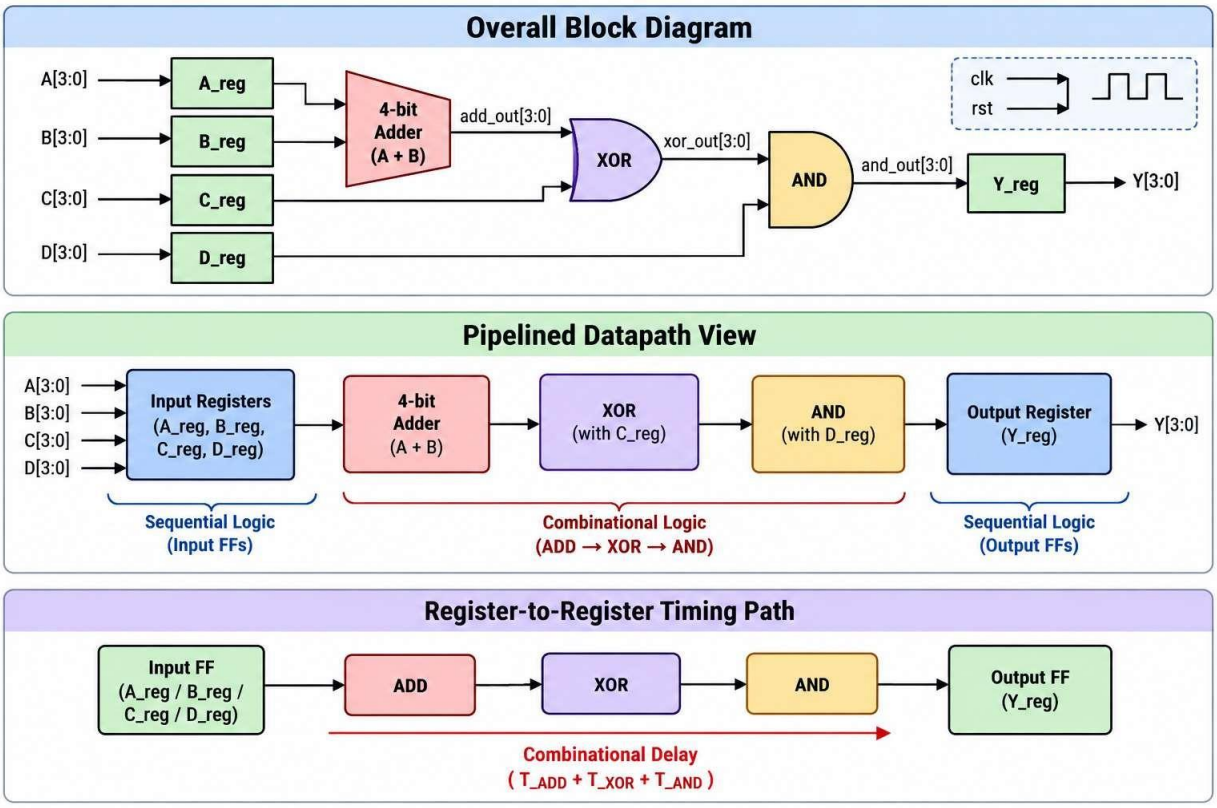
VERILOG CODE:

The following editable transcription preserves the supplied logic and synchronous reset. Related declarations are grouped for readability

```
module setup_analysis (
input wire clk, rst,
    input wire [3:0] A, B, C, D,
    output reg [3:0] Y
);
    reg [3:0] A_reg, B_reg, C_reg, D_reg;
    wire [3:0] add_out, xor_out, and_out;
    assign add_out = A_reg + B_reg;
    assign xor_out = add_out ^ C_reg;
    assign and_out = xor_out & D_reg;
    always @(posedge clk) begin
        if (rst) begin
            A_reg <= 4'b0000;
            B_reg <= 4'b0000;
            C_reg <= 4'b0000;
            D_reg <= 4'b0000;
        end  else  begin
            A_reg <= A;
            B_reg <= B;
            C_reg <= C;
            D_reg <= D;
        end
    end
    always @(posedge clk) begin
        if (rst) Y <= 4'b0000;
        else     Y <= and_out;
    end
endmodule
```

CODE EXPLANATION:

Continuous assign statements describe the combinational path. The first always block stores all four inputs or resets them to zero. The second always block stores and_out in Y or clears Y. Both blocks respond to the same rising clock edge and use nonblocking assignments, preserving the previous input-register values for the current output update.



SOURCE CODE PHOTO:

Project Summary | Device | setup_analysis.v

D:/Arush/exp3setup_analysis/exp3setup_analysis.srcs/sources_1/new/setup_analysis.v

```
23  module setup_analysis (
24   input wire clk,
25   input wire rst,
26   input wire [3:0] A,
27   input wire [3:0] B,
28   input wire [3:0] C,
29   input wire [3:0] D,
30   output reg [3:0] Y
31  );
32   reg [3:0] A_reg;
33   reg [3:0] B_reg;
34   reg [3:0] C_reg;
35   reg [3:0] D_reg;
36   wire [3:0] add_out;
37   wire [3:0] xor_out;
38   wire [3:0] and_out;
39  // Combinational logic
40   assign add_out = A_reg + B_reg;
41   assign xor_out = add_out ^ C_reg;
42   assign and_out = xor_out & D_reg;
43   // Input registers
44   always @(posedge clk) begin
45   if (rst) begin
46   A_reg <= 4'b0000;
47   B_reg <= 4'b0000;
48   C_reg <= 4'b0000;
49   D_reg <= 4'b0000;
50   end
51   else begin
52   A_reg <= A;
53   B_reg <= B;
54   C_reg <= C;
55   D_reg <= D;
56   end
57   end
58   // Output register
59   always @(posedge clk) begin
60   if (rst)
61   Y <= 4'b0000;
62   else
63   Y <= and_out;
64   end
65  endmodule
66
```

Figure 3.1: Supplied setup_analysis Verilog design source.

DESIGN AND EXPECTED RESULTS:

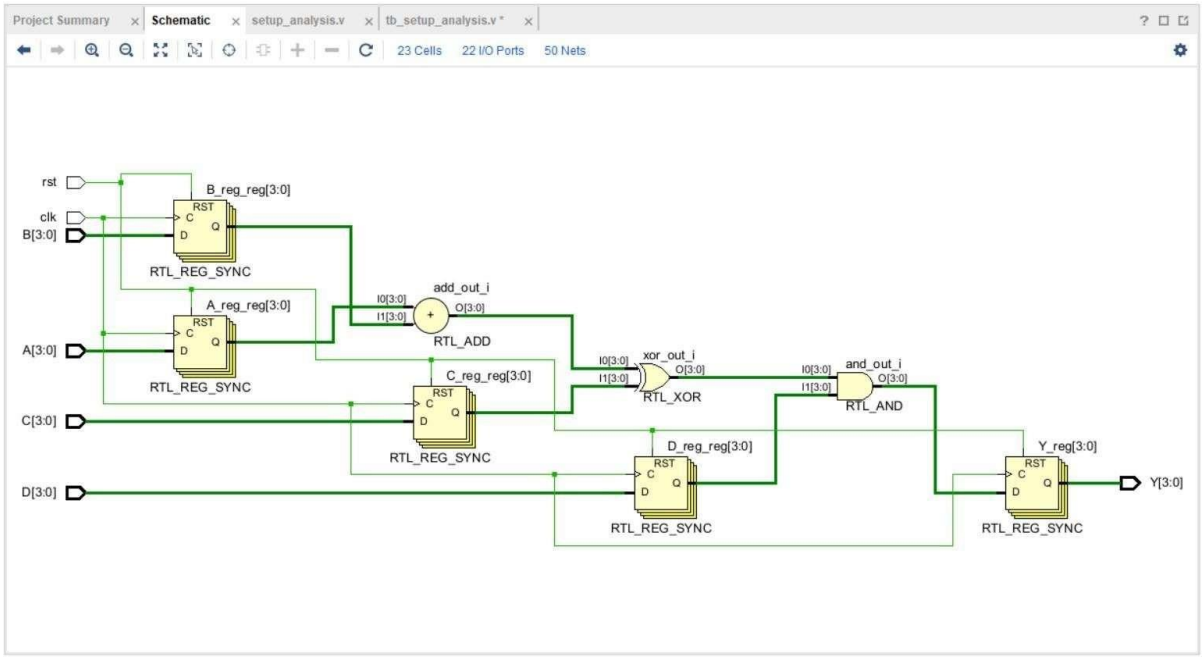


Figure 3.3: Supplied RTL schematic: input registers, ADD, XOR, AND and output register.

DESIGN EXPLANATION:

The schematic shows four banks of synchronous input registers feeding an adder, XOR gate and AND gate. A final register bank captures Y. The four-bit buses carry all four bits in parallel. clk controls every register bank and rst clears them synchronously.

TESTBENCH CODE:

This editable transcription follows the supplied test sequence. An explicit `timescale 1ns/1ps directive is added so #5 represents 5 ns and the clock period is 10 ns.

```
`timescale 1ns/1ps

module tb_setup_analysis;
   reg clk, rst;

   reg [3:0] A, B, C, D;

   wire [3:0] Y;
   setup_analysis DUT (

      .clk(clk), .rst(rst),

      .A(A), .B(B), .C(C), .D(D), .Y(Y)

   );
   always #5 clk = ~clk;
   initial begin

      clk = 0; rst = 1;

      A = 4'd0; B = 4'd0;
      C = 4'd0; D = 4'd0;
      #20;

      rst = 0;

      A = 4'd5; B = 4'd3;
      C = 4'd2; D = 4'd15;
      #20;

      A = 4'd7; B = 4'd2;
      C = 4'd1; D = 4'd10;
      #20;

      A = 4'd9; B = 4'd4;
      C = 4'd3; D = 4'd12;
      #20;

      $finish;
   end

endmodule
```

TESTBENCH EXPLANATION:

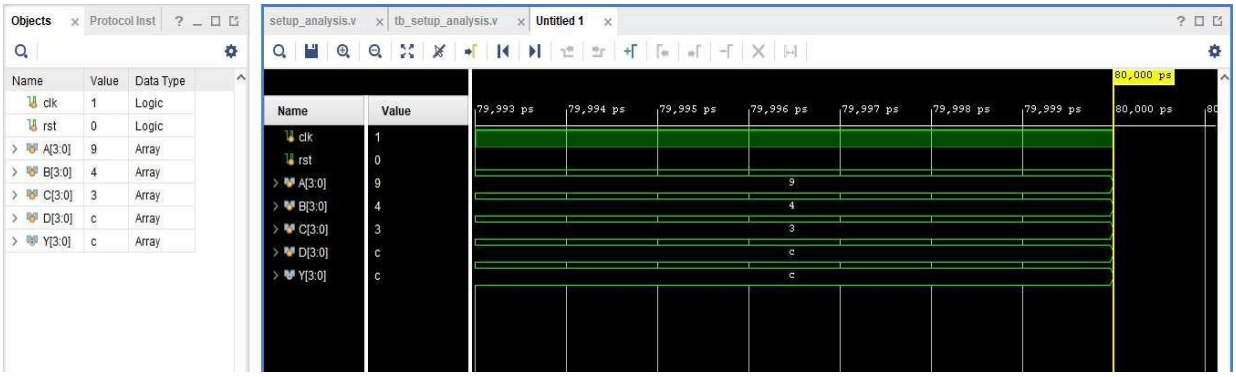
DUT means Design Under Test. Named connections link the testbench signals to the design ports. The clock toggles every 5 ns. Reset is held high for 20 ns, followed by three input sets, each held for 20 ns. The simulation stops at 80 ns. Inputs change on falling edges here, avoiding a race with rising-edge register updates.

TESTBENCH PHOTO:

```
module tb_setup_analysis;
 reg clk;
 reg rst;
 reg [3:0] A;
 reg [3:0] B;
 reg [3:0] C;
 reg [3:0] D;
 wire [3:0] Y;
 setup_analysis DUT (
 .clk(clk),
 .rst(rst),
 .A(A),
 .B(B),
 .C(C),
 .D(D),
 .Y(Y)
 );
 // 10 ns clock period
 always #5 clk = ~clk;
 initial begin
 clk = 0;
 rst = 1;
 A = 4'd0;
 B = 4'd0;
 C = 4'd0;
 D = 4'd0;
 #20;
 rst = 0;
 A = 4'd5;
 B = 4'd3;
 C = 4'd2;
 D = 4'd15;
 #20;
 A = 4'd7;
 B = 4'd2;
 C = 4'd1;
 D = 4'd10;
 #20;
 A = 4'd9;
 B = 4'd4;
 C = 4'd3;
 D = 4'd12;
 #20;
 $finish;
 end
 endmodule
```

Figure 3.2: Supplied testbench source (not a simulation waveform).

WAVE FORM :-



SIMULATION RESULTS:

Input applied	A, B, C, D	Sum / XOR / AND	Y valid from
20 ns	5, 3, 2, 15	8 / 10 / 10	35 ns: 10 (1010)
40 ns	7, 2, 1, 10	9 / 8 / 8	55 ns: 8 (1000)
60 ns	9, 4, 3, 12	13 / 14 / 12	75 ns: 12 (1100)

For example, the first set is captured at 25 ns. Its combinational result becomes 10 after that capture, and Y stores it at 35 ns. Y remains 0 at 25 ns because it still uses the previously reset input registers. At 45 ns Y is still 10; at 65 ns it is still 8.

RESULT:

The supplied code and RTL schematic show the intended four-bit registered arithmetic/logic datapath. Expected outputs for the supplied test sequence are 10, 8 and 12 after the register delay. These values are calculated from the code, not presented as observed Vivado measurements. The simulation waveform screenshot is pending.

EXPERIMENT NO. 4

AIM

To perform Hold Time Analysis and fix Hold Time Violation using Buffer Insertion in Vivado.

TOOLS REQUIRED

PC/Laptop, Xilinx Vivado Design Suite, Verilog HDL, and XDC Constraint File.

INTRODUCTION

Hold time is the minimum amount of time for which the data input of a capture flip-flop must remain stable after the active clock edge. In Static Timing Analysis (STA), hold analysis checks the minimum-delay path between a launch flip-flop and a capture flip-flop.

A hold violation occurs when new data reaches the capture flip-flop too early. One common method to fix a hold violation is to increase the minimum data-path delay by inserting buffers/delay elements.

NUMERICAL VALUES

Parameter	Value
Clock Period	10 ns
Clock Frequency	100 MHz
Clock-to-Q minimum	Approximately 0.1 ns
Data path minimum delay	Approximately 0.1 ns
Hold time of capture FF	0.3 ns
Hold uncertainty	0.0 ns initially

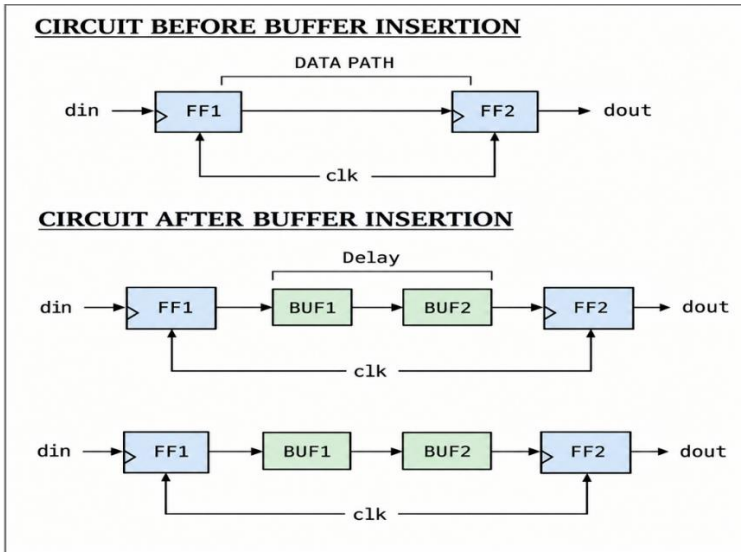
THEORY

For hold analysis, the important timing path is:

FF1/Q ——————————► FF2/D

If the data reaches the capture flip-flop before the required hold time is satisfied, hold slack becomes negative and a hold violation occurs.

To fix a hold violation, delay is added to the minimum-delay data path. In this experiment, two delay elements are inserted between the launch and capture flip-flops.



PROCEDURE

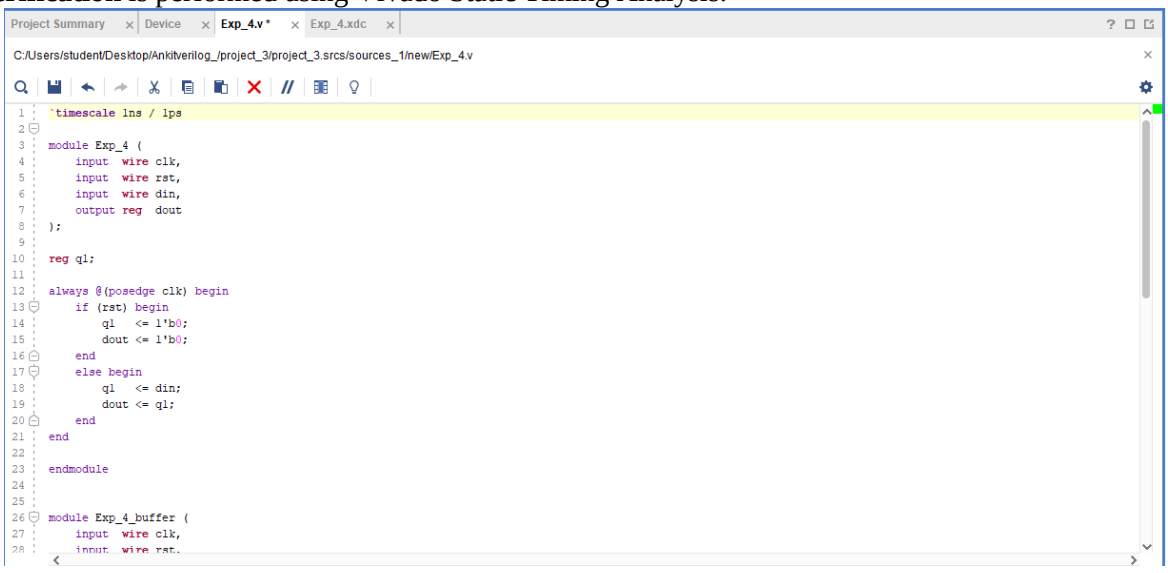
1. Open Xilinx Vivado and create a new RTL project.
2. Create the Verilog source file hold_test.v.
3. Enter the initial design without buffers and set it as the top module.
4. Run synthesis and open the synthesized design.
5. Create the XDC constraint file hold_test.xdc.
6. Apply the 10 ns clock constraint using create_clock.
7. Run timing analysis and inspect the minimum-delay/hold path.
8. Identify the path from q1/Q to dout/D.
9. Create the buffered design hold_test_buffer.v.
10. Insert two delay elements between q1 and dout.
11. Run synthesis again and perform minimum-delay timing analysis.
12. Compare the hold timing before and after buffer insertion.

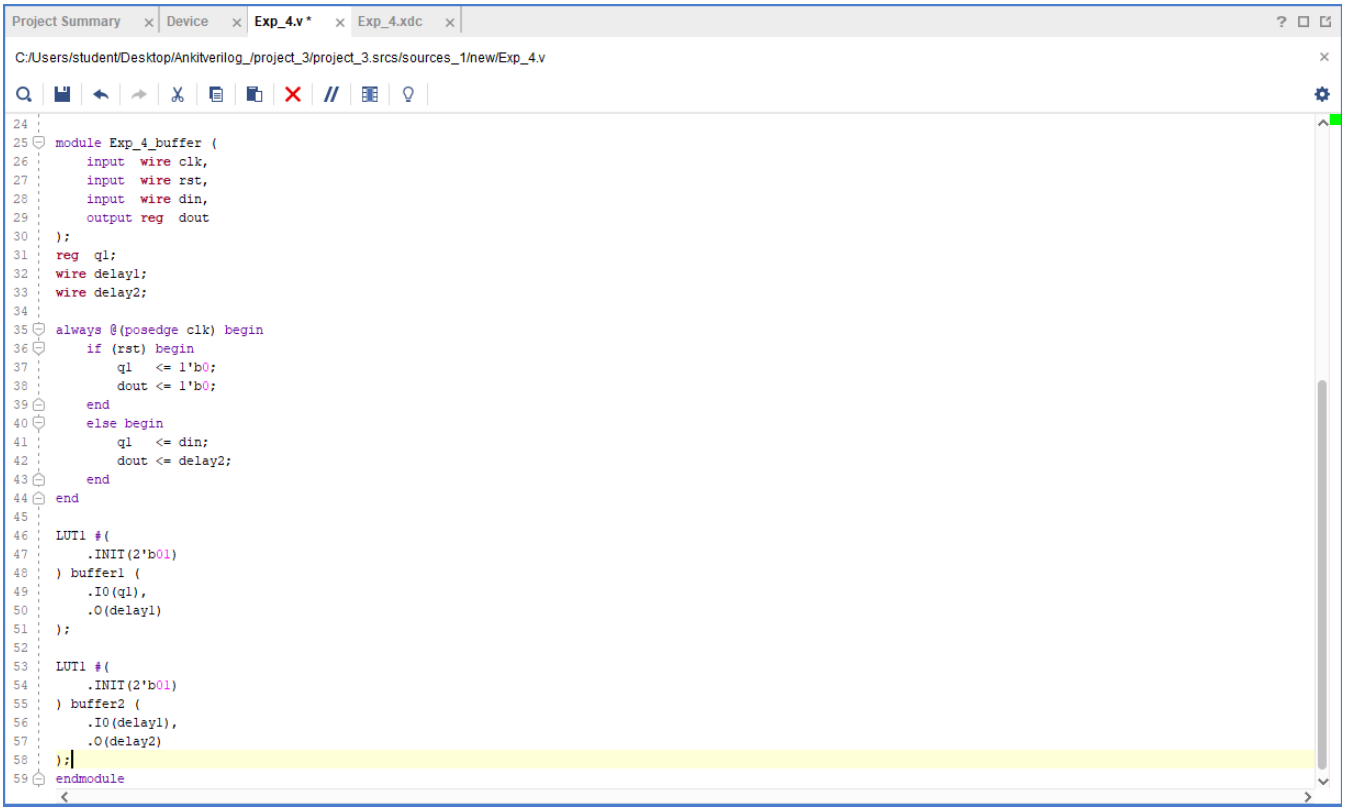
VERILOG CODE

```
module clock_skew_uncertainty (
   input  wire clk,
   input  wire reset,
   input  wire d,
   output reg  q);
   reg q1;
   always @(posedge clk or posedge reset) begin
      if (reset) begin
         q1 <= 1'b0;
         q  <= 1'b0;
  end
      else begin
         q1 <= d;
         q  <= q1;
      end
   end
endmodule
```

Testbench Explanation:-

The **testbench** is used to simulate and verify the behavior of the Hold Time Analysis circuit. The actual **hold-time verification** is performed using Vivado Static Timing Analysis.





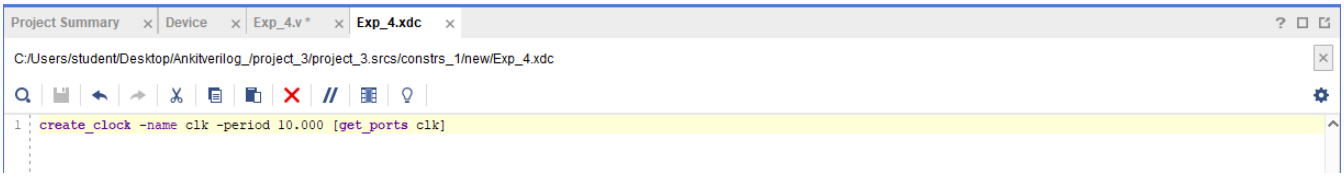
Create XDC Constraint:-

Create:

Exp_4.xdc

Put:

create_clock -name clk -period 10.000 [get_ports clk]



Run Synthesis and Timing Analysis:-

After creating the XDC constraint file, save the complete Vivado project and run synthesis. In Vivado, go to **Flow Navigator → Synthesis → Run Synthesis**. After synthesis is completed successfully, select **Open Synthesized Design**. Then go to **Reports → Timing Summary** to view the timing results. The timing summary can also be generated using the Tcl command:

report_timing_summary

For minimum-delay and hold-time analysis, use the following Tcl command in the Vivado Tcl Console:

report_timing -delay_type min

The important timing path to observe is:

Startpoint: q1/Q

Endpoint: dout/D

The path under analysis is:

FF1/Q ——————————► FF2/D

After inserting the two delay elements, the path becomes:

q1/Q ⟶ buffer1 ⟶ buffer2 ⟶ dout/D

Observe the **minimum delay, hold requirement, and hold slack** in the timing report. Compare the timing results before and after buffer insertion. The purpose of buffer insertion is to increase the minimum data-path delay and improve the hold-timing condition.

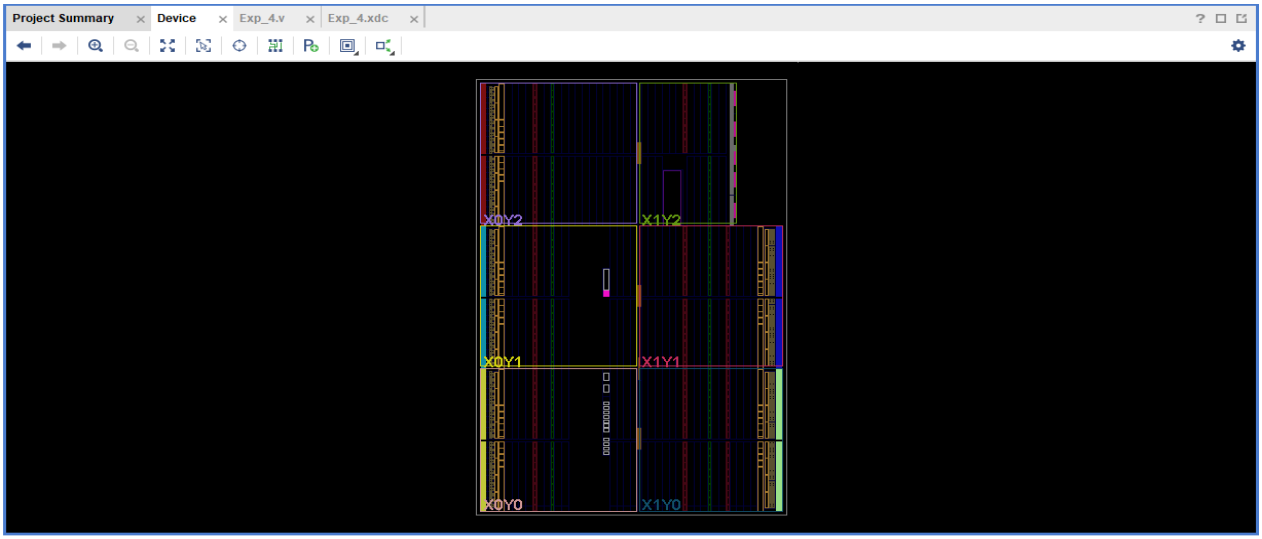
Tcl Commands:

report_timing_summary

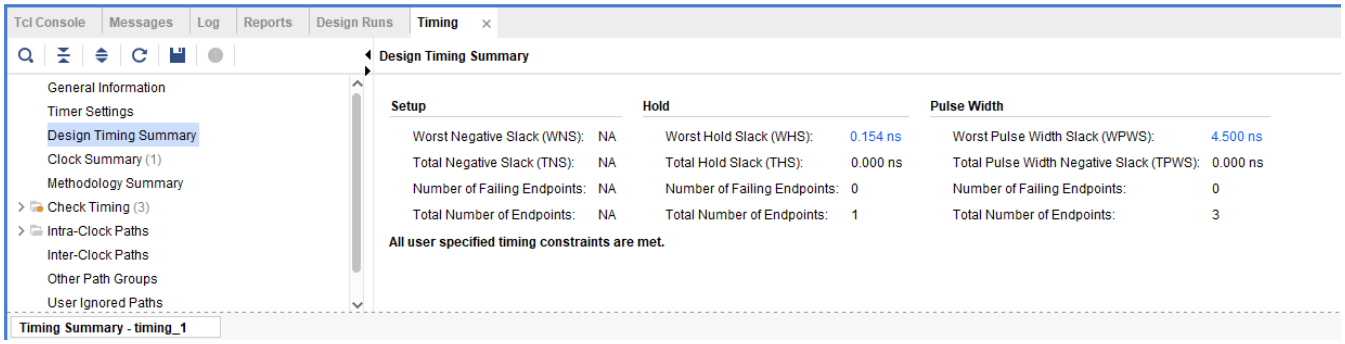
report_timing -delay_type min

Important: The exact hold-slack values should be taken from the Vivado timing report obtained during the experiment.

DEVICE:-



OUTPUT:-



CONCLUSION:-

In this experiment, **Hold Time Analysis and Hold Violation Fixing using Buffer Insertion** were performed. The minimum-delay timing path from **FF1/Q to FF2/D** was analyzed. Two buffers were inserted in the data path to increase the minimum delay. It was observed that buffer insertion improves the hold timing of the circuit and helps in reducing hold-time violations.

EXPERIMENT-5

AIM:-

To simulate and analyze the effect of **clock skew** and **clock uncertainty** on the timing performance of a synchronous digital circuit using **Xilinx Vivado**.

Required Software:

- Xilinx Vivado
- Verilog HDL
- Vivado Timing Analysis / Simulation

Theory:

Clock Skew:

Clock skew is the difference in the arrival time of the same clock edge at the launch and capture flip-flops.

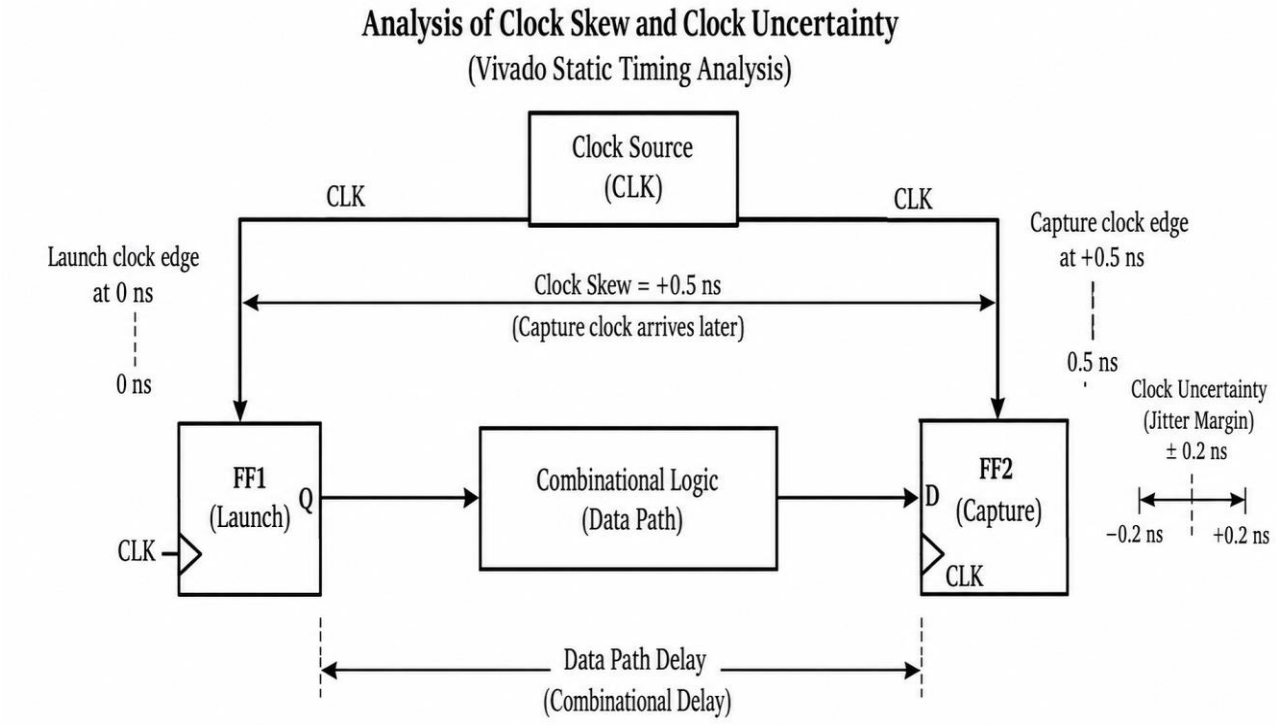
Clock Skew=Tcapture − TlaunchClock

Clock Uncertainty:

Clock uncertainty represents the timing margin required because of clock jitter, variation, and other clock-related uncertainties.

In Vivado, it can be specified using:

set_clock_uncertainty 0.2 [get_clocks clk]



PROCEDURE:-

1.Open Vivado and create a new RTL project.

2.Add the Verilog source code clock_skew_uncertainty.v to the project.

3.Set the Verilog module as the Top Module.

4.Create an XDC constraint file.

5Add clock uncertainty.

6.Run Synthesis.

7.Run Implementation.

8.Open:

Open Implemented Design

↓

Report Timing Summary

9.Select:

Reports → Timing → Report Timing

10.For detailed setup timing, use:

report_timing -delay_type max

11.For detailed hold timing, use:

report_timing -delay_type min

12.Observe the following parameters:

- Data path delay
- Clock arrival time
- Clock skew
- Clock uncertainty
- Setup slack
- Hold slack

13.Change the clock uncertainty, for example:

set_clock_uncertainty 0.5 [get_clocks clk]

14.Run timing analysis again and compare the setup and hold slack before and after changing the uncertainty.

VERILOG CODE:-

```
module clock_skew_uncertainty (
   input  wire clk,
   input  wire reset,
   input  wire d,
   output reg  q
);
   reg q1;
   always @(posedge clk or posedge reset) begin
      if (reset) begin
         q1 <= 1'b0;
         q  <= 1'b0;
      end
      else begin
         q1 <= d;
         q  <= q1;
      end
   end
endmodule
```

Project Summary | Device | exp5.v | exp5.xdc

D:/New folder/project_3/project_3.srcs/sources_1/new/exp5.v

```
`timescale 1ns / 1ps
module clock_skew_uncertainty (
    input  wire clk,
    input  wire reset,
    input  wire d,
    output reg  q
);

    reg q1;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            q1 <= 1'b0;
            q  <= 1'b0;
        end
        else begin
            q1 <= d;
            q  <= q1;
        end
    end

endmodule
```

Vivado XDC Constraints:-

Create an .xdc file and add:

create_clock -name clk -period 10.000 [get_ports clk]

set_clock_uncertainty 0.2 [get_clocks clk]

Project Summary | Device | exp5.v | exp5.xdc

D:/New folder/project_3/project_3.srcs/constrs_1/new/exp5.xdc

```
create_clock -name clk -period 10.000 [get_ports clk]

set_clock_uncertainty 0.2 [get_clocks clk]

```

Here:

Clock period = 10 ns

Clock frequency = 100 MHz

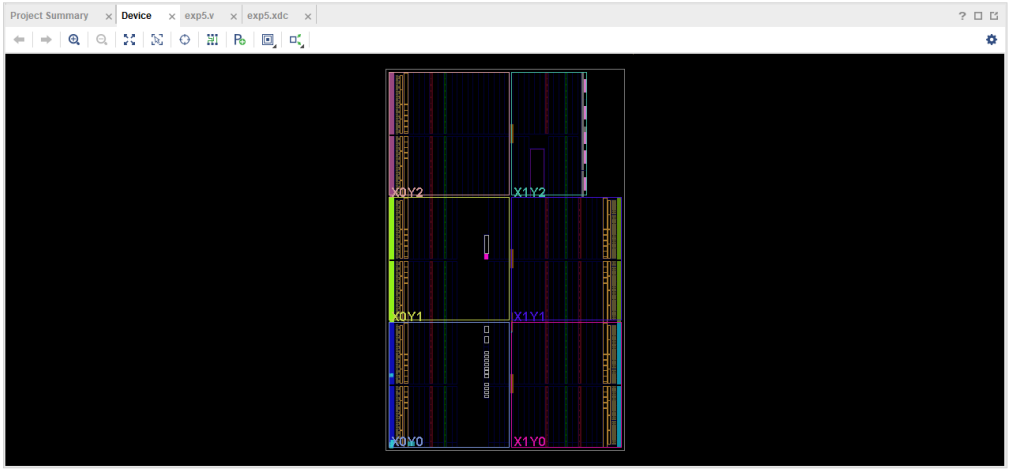
Clock uncertainty = 0.2 ns

To Observe Timing

After synthesis/implementation:

Vivado → Open Implemented Design

DEVICE:-



Vivado → Open Implemented Design → Report Timing Summary
Then select:
Reports → Timing → Report Timing
You can also use Tcl:
report_timing_summary
For detailed setup timing:
report_timing -delay_type max
For hold timing:
report_timing -delay_type min

OUTPUT:-

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	8.396 ns	Worst Hold Slack (WHS):	0.192 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	1	Total Number of Endpoints:	1	Total Number of Endpoints:	3

All user specified timing constraints are met.

CONCLUSION:-

The effect of clock skew and clock uncertainty on the timing performance of a synchronous digital circuit was successfully analyzed using Xilinx Vivado. It was observed that increasing clock uncertainty affects the setup and hold slack, thereby influencing the overall timing performance of the circuit.

EXPERIMENT:-6

AIM:

To Simulate the timing Exception Analysis using set_false_path.

REQUIRED SOFTWARE:

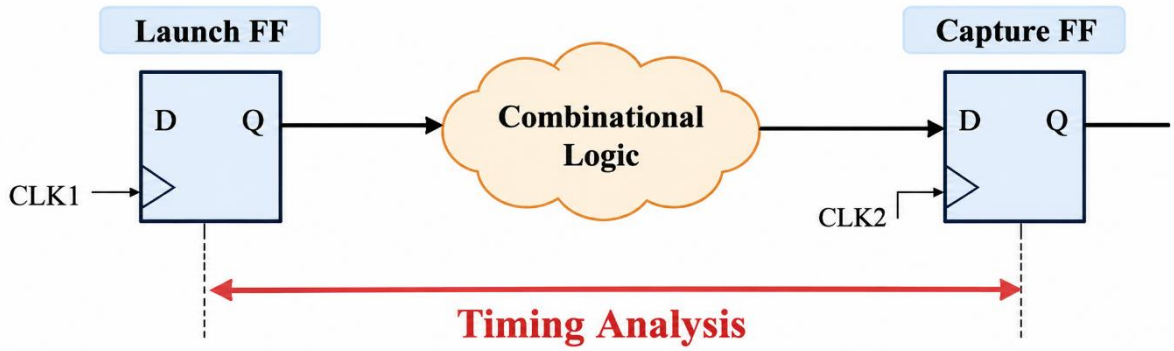
- Xilinx Vivado
- Verilog HDL
- Vivado Timing Analysis / Simulation

THEORY:

Normally, Vivado checks timing between launch and capture points across flip-flops by assuming that all clocks are synchronous. It verifies setup time and hold time requirements for all register-to-register paths.

In practical designs, circuits frequently use multiple clock domains that are asynchronous to each other (i.e., having no fixed phase relationship, such as clk_a = 100 MHz and clk_b = 80 MHz). In such cases, standard setup and hold timing checks across the asynchronous boundaries are physically meaningless and produce unwanted timing violations.

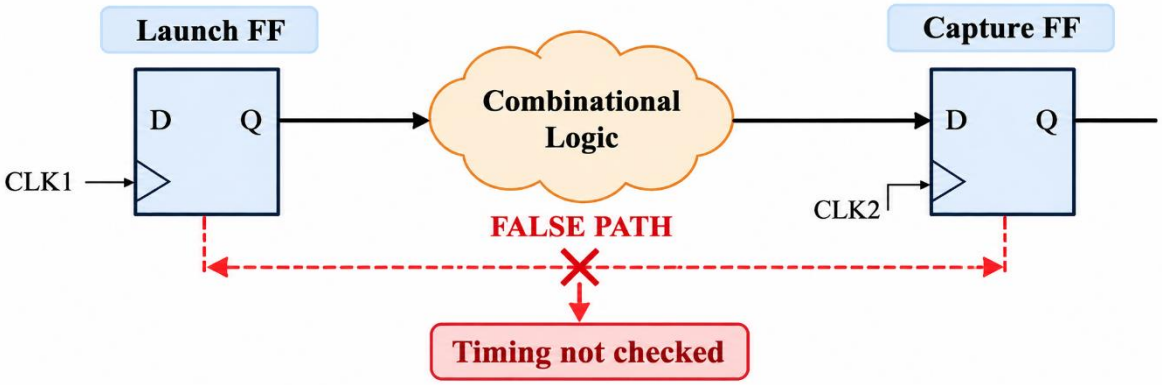
After applying the set_false_path constraint, Vivado ignores the specified timing paths during Static Timing Analysis (STA). This instructs the implementation and timing engine not to perform setup/hold verification on those paths, avoiding unfixable timing violations and preventing optimization overhead on non-critical asynchronous logic.



After applying:

set_false_path

Vivado ignores the specified timing path



PROCEDURE:

1. Open Xilinx Vivado and create a new RTL project.
2. Add the Verilog source file false_path_demo.v to the project.
3. Set the Verilog module as the Top Module.
4. Create an XDC constraint file.
5. Define the two clocks using create_clock.
6. Initially, perform Static Timing Analysis (STA) without applying the false-path constraint.
7. Run Synthesis.
8. Run Implementation.
9. Open Implemented Design.
10. Select Reports → Timing → Report Timing Summary.
11. Use the Tcl command report_timing_summary if required.
12. Analyze the timing path using report_timing.
13. Observe the timing path from data_a_reg/Q through routing to data_b_reg/D.
14. Add the false-path constraint: set_false_path -from [get_clocks clk_a] -to [get_clocks clk_b].
15. Reset the implementation run.
16. Run Synthesis again.
17. Run Implementation again.
18. Open Implemented Design.
19. Open Reports → Timing → Report Timing Summary.
20. Compare the timing analysis before and after applying set_false_path.
21. Verify the applied timing exception using report_exceptions.
22. Observe that Vivado no longer considers the specified path during normal timing analysis.

VERILOG CODE:

```
module false_path_demo (
   input  wire clk_a,
   input  wire clk_b,
   input  wire reset,
   input  wire data_in,
   output reg  data_out);
   reg data_a;
   reg data_b;
   always @(posedge clk_a or posedge reset) begin
      if (reset)
         data_a <= 1'b0;
      else
         data_a <= data_in;
   end
   always @(posedge clk_b or posedge reset) begin
    if (reset)
         data_b <= 1'b0;
      else
```

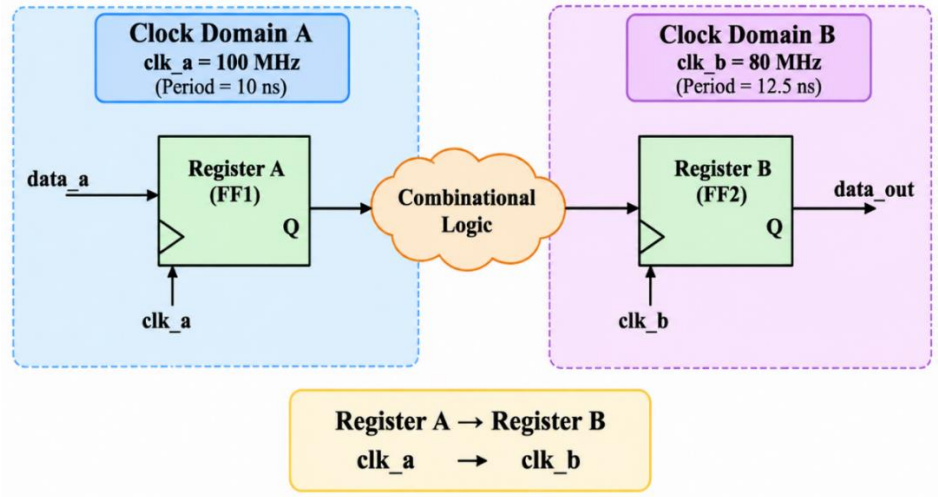
```
        data_b <= data_a;
    end
    assign data_out = data_b;
endmodule
```

Project Summary | Device | exp_6.v | exp_6_xdc.xdc

C:/Users/student/Downloads/SATYAM/exp_6/exp_6.srcs/sources_1/new/exp_6.v

```
1  module false_path_demo (
2      input wire clk_a,
3      input wire clk_b,
4      input wire reset,
5      input wire data_in,
6      output wire data_out
7  );
8
9  reg data_a;
10 reg data_b;
11
12 always @(posedge clk_a or posedge reset) begin
13     if (reset)
14         data_a <= 1'b0;
15     else
16         data_a <= data_in;
17 end
18
19 always @(posedge clk_b or posedge reset) begin
20     if (reset)
21         data_b <= 1'b0;
22     else
23         data_b <= data_a;
24 end
25
26 assign data_out = data_b;
27
28 endmodule
```

data_a is launched by clk_a.
Then it is captured by clk_b.
So the path is:

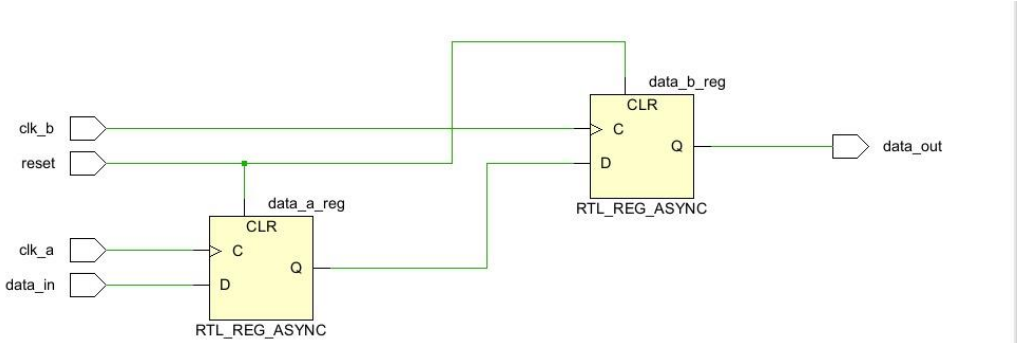


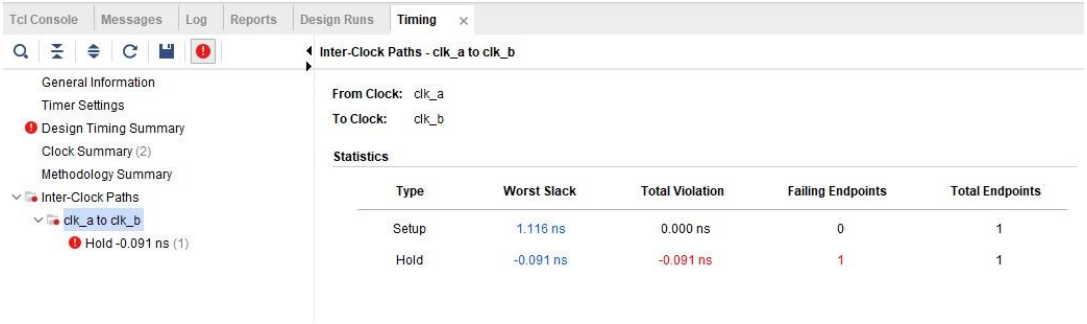
XDC CONSTRAINT:

create_clock -name clk_a -period 10.000 [get_ports clk_a]
create_clock -name clk_b -period 12.500 [get_ports clk_b]

BLOCK DIAGRAM DESCRIPTION:

Clock Domain 1: Register A (FF1) operating at clk_a = 100 MHz (Period = 10 ns).
Combinational / Routing Path: Path transferring data_a from Register A to Register B.
Clock Domain 2: Register B (FF2) operating at clk_b = 80 MHz (Period = 12.5 ns).
Timing Exception: Applying set_false_path breaks the timing check between clk_a and clk_b (Timing not checked).





STA WITHOUT FALSE PATH

Initially, do not add the set_false_path constraint.

Run: Flow Navigator → Run Synthesis → Run Implementation.

Then: Open Implemented Design → Reports → Timing → Report Timing Summary.

```
report_timing_summary
report_timing
```

Vivado normally attempts to analyze the path:

```
data_a_reg/Q
      ↓
   Routing
      ↓
data_b_reg/D
```

APPLY FALSE PATH CONSTRAINT

```
set_false_path -from [get_clocks clk_a] -to [get_clocks clk_b]
```

This means: Ignore timing paths launched by clk_a and captured by clk_b.

For both directions, if required:

```
set_false_path -from [get_clocks clk_a] -to [get_clocks clk_b]
set_false_path -from [get_clocks clk_b] -to [get_clocks clk_a]
```

For this experiment, the first constraint is sufficient because the required path is from clk_a to clk_b.

RE-RUN TIMING ANALYSIS

After modifying the XDC file:

Reset Implementation Run → Run Synthesis → Run Implementation → Open Implemented Design → Reports → Timing Summary

```
report_timing_summary
```

After applying the false-path constraint, Vivado no longer considers the specified clk_a → clk_b path during normal timing analysis.

IMPORTANT VIVADO TCL COMMANDS:-

Create Clocks

```
create_clock -name clk_a -period 10 [get_ports clk_a]
create_clock -name clk_b -period 12.5 [get_ports clk_b]
```

Check Clocks

```
report_clocks
```

Check Timing

```
report_timing
```

Timing Summary

```
report_timing_summary
```

Apply False Path

```
set_false_path -from [get_clocks clk_a] -to [get_clocks clk_b]
```

Check Timing Exceptions

```
report_exceptions
```

DEVICE:-

Device: Select the device/board specified by your Vivado project.

After implementation: Vivado → Open Implemented Design → Report Timing Summary.

Then select: Reports → Timing → Report Timing.

OUTPUT:-

Before applying set_false_path:

```
clk_a
  ↓
data_a_reg
  ↓
Routing
  ↓
data_b_reg
  ↓
clk_b
```

After applying set_false_path:

```
clk_a ——————————————X——————————————> clk_b
                    FALSE PATH
```

The specified timing path is excluded from normal timing analysis.

```
Timing Report

Slack (MET) :             1.116ns  (required time - arrival time)
  Source:                 data_a_reg/C
                            (rising edge-triggered cell FDCE clocked by clk_a  {rise@0.000ns fall@5.000ns period=10.000ns})
  Destination:            data_b_reg/D
                            (rising edge-triggered cell FDCE clocked by clk_b  {rise@0.000ns fall@6.250ns period=12.500ns})
  Path Group:             clk_b
  Path Type:              Setup (Max at Slow Process Corner)
  Requirement:            2.500ns  (clk_b rise@12.500ns - clk_a rise@10.000ns)
  Data Path Delay:        0.790ns  (logic 0.456ns (57.722%)  route 0.334ns (42.278%))
  Logic Levels:           0
  Clock Path Skew:        -0.323ns (DCD - SCD + CPR)
    Destination Clock Delay (DCD):    2.092ns = ( 14.592 - 12.500 )
    Source Clock Delay      (SCD):    2.415ns = ( 12.415 - 10.000 )
    Clock Pessimism Removal (CPR):    0.000ns
  Clock Uncertainty:      0.035ns  ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
    Total System Jitter     (TSJ):    0.071ns
    Total Input Jitter      (TIJ):    0.000ns
    Discrete Jitter          (DJ):    0.000ns
    Phase Error              (PE):    0.000ns
  Clock Domain Crossing:  Inter clock paths are considered valid unless explicitly excluded by timing constraints such as set_clock_groups or set_false_path.
```

Location	Delay type	Incr(ns)	Path(ns)		Netlist Resource(s)
	(clock clk_a rise edge)	10.000	10.000	r	
		0.000	10.000	r	clk_a (IN)
	net (fo=0)	0.000	10.000		clk_a
	IBUF (Prop_ibuf_I_O)	0.936	10.936	r	clk_a_IBUF_inst/O
	net (fo=1, unplaced)	0.800	11.735		clk_a_IBUF
	BUFG (Prop_bufg_I_O)	0.096	11.831	r	clk_a_IBUF_BUFG_inst/O
	net (fo=1, unplaced)	0.584	12.415		clk_a_IBUF_BUFG
	FDCE			r	data_a_reg/C
	FDCE (Prop_fdce_C_Q)	0.456	12.871	r	data_a_reg/Q
	net (fo=1, unplaced)	0.334	13.205		data_a
	FDCE			r	data_b_reg/D
	(clock clk_b rise edge)	12.500	12.500	r	
		0.000	12.500	r	clk_b (IN)
	net (fo=0)	0.000	12.500		clk_b
	IBUF (Prop_ibuf_I_O)	0.802	13.302	r	clk_b_IBUF_inst/O
	net (fo=1, unplaced)	0.760	14.062		clk_b_IBUF
	BUFG (Prop_bufg_I_O)	0.091	14.153	r	clk_b_IBUF_BUFG_inst/O
	net (fo=1, unplaced)	0.439	14.592		clk_b_IBUF_BUFG
	FDCE			r	data_b_reg/C
	clock pessimism	0.000	14.592		
	clock uncertainty	-0.035	14.557		
	FDCE (Setup_fdce_C_D)	-0.235	14.322		data_b_reg
	required time		14.322		
	arrival time		-13.205		
	slack		1.116		

CONCLUSION:-

The timing exception using the set_false_path constraint was successfully implemented and analyzed using Xilinx Vivado. The timing path between the asynchronous clock domains clk_a and clk_b was initially analyzed by Vivado. After applying the set_false_path constraint, the specified timing path was ignored during normal timing analysis. Thus, the experiment demonstrated how false-path constraints are used to exclude paths that do not have a valid timing relationship from STA.

EXPERIMENT NO. 7

AIM: To simulate Multicycle Path Timing Analysis using the set_multicycle_path constraint in Xilinx Vivado.

REQUIRED SOFTWARE: Xilinx Vivado Design Suite, Verilog HDL, and an XDC constraint file.

THEORY:

Static Timing Analysis (STA) checks whether data launched by one register reaches a capture register within the required timing window. By default, a synchronous register-to-register path is checked against a single clock period. A multicycle path is a path for which the design intentionally allows data more than one clock cycle to reach the destination register.

The set_multicycle_path constraint modifies the setup timing requirement for the selected path. In this experiment, a 10 ns clock is used, corresponding to 100 MHz. A two-cycle setup constraint allows approximately 20 ns for setup analysis on the selected path instead of the normal 10 ns. A corresponding hold adjustment is added so that the hold check remains appropriate.

A multicycle constraint must only be applied when the design's functional behaviour genuinely permits the extra cycle. Applying it broadly to every path on a clock can hide real timing problems; in a practical design, select the intended startpoint and endpoint registers.

DESIGN DESCRIPTION

The circuit contains two sequential registers. The first register, d_reg, captures input d at the rising edge of clk. The second register, q, captures the previous value of d_reg. Reset rst is synchronous and active high. The register-to-register path from d_reg to q is used to demonstrate how the timing requirement changes when a multicycle exception is applied.

Timing path: d_reg/Q ——————————► q/D

Clock period = 10 ns; clock frequency = 1 / 10 ns = 100 MHz.

VERILOG CODE (multicycle_demo.v)

```
module multicycle_demo (
input wire clk,
input wire rst,
input wire d,
output reg q);
reg d_reg;
always @(posedge clk) begin
if (rst) begin
d_reg <= 1'b0;
q <= 1'b0;
end
else begin
d_reg <= d;
q <= d_reg;
end
end
endmodule
```

XDC CONSTRAINTS (multicycle_demo.xdc)

```
create_clock -name clk -period 10.000 [get_ports clk]
# Two-cycle setup requirement for the selected demonstration path
set_multicycle_path 2 -setup \
-from [get_clocks clk] \
-to [get_clocks clk]
# Corresponding hold adjustment
set_multicycle_path 1 -hold \
-from [get_clocks clk] \
-to [get_clocks clk]
```

Note: The clock-based example above affects same-clock paths generally. For a real design, replace the broad clock selectors with the specific launch and capture registers, using the synthesized cell names reported by Vivado.

PROCEDURE

1. Open Xilinx Vivado and create a new RTL project.
2. Add multicycle_demo.v under Design Sources and set the module as the design top.
3. Create an XDC constraint file and define the 10 ns clock using create_clock.
4. Run Synthesis, then Run Implementation.
5. Open Implemented Design and generate Reports → Timing → Report Timing Summary.
6. Record the normal setup requirement and inspect the register-to-register path using report_timing.
7. Add the set_multicycle_path 2 -setup constraint for the intended path and the corresponding set_multicycle_path 1 -hold adjustment.
8. Re-run implementation and generate the timing report again.
9. Compare the setup requirement, data path delay, WNS, and TNS before and after applying the multicycle constraint.
10. Use report_exceptions to verify that Vivado recognizes the applied timing exception.

Project Summary | Device | multicycle_demo.v | multicycle_demo.xdc

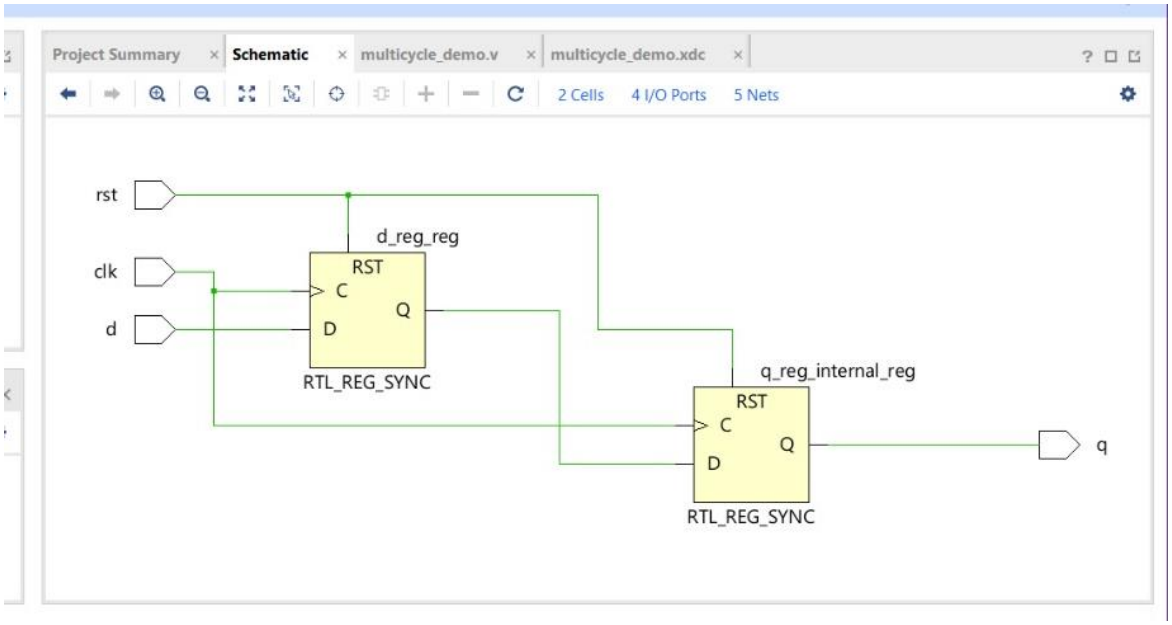
C:/Users/Hp/FPGA_PROJECTS_II/project_1/project_1.srcs/sources_1/new/multicycle_demo.v

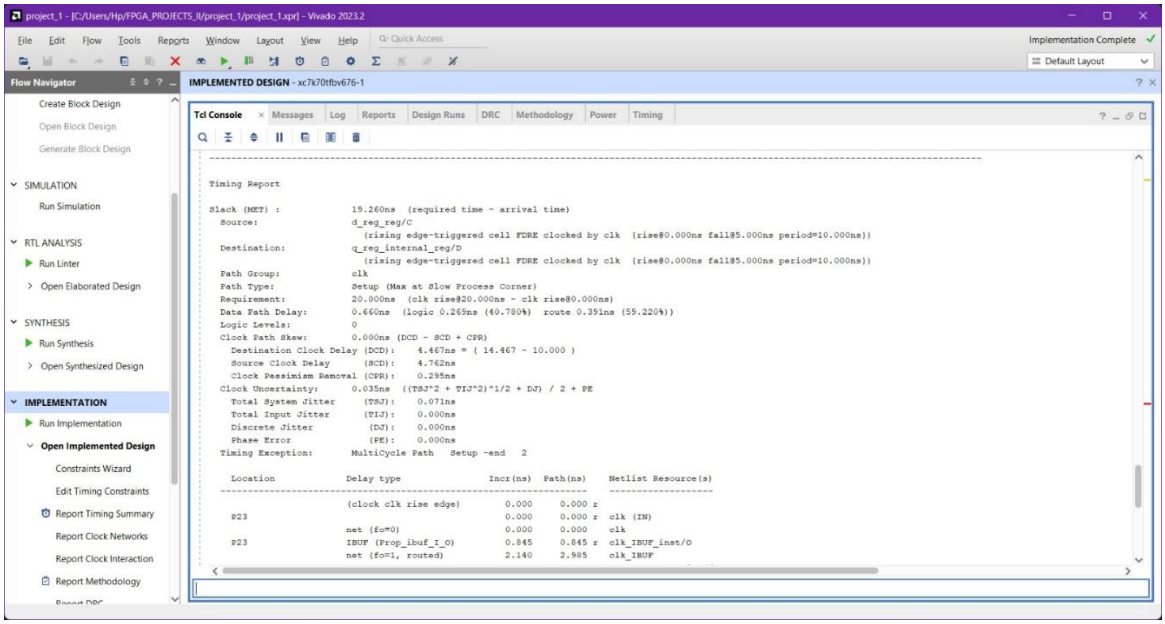
```
1  `timescale 1ns / 1ps
2
3  module multicycle_demo (
4      input  wire clk,
5      input  wire rst,
6      input  wire d,
7      output reg  q
8  );
9
10     (* dont_touch = "true" *) reg d_reg;
11     (* dont_touch = "true" *) reg q_reg_internal;
12
13     always @(posedge clk) begin
14         if (rst) begin
15             d_reg          <= 1'b0;
16             q_reg_internal <= 1'b0;
17         end else begin
18             d_reg          <= d;
19             q_reg_internal <= d_reg;
20         end
21     end
22
23     always @(*) begin
24         q = q_reg_internal;
25     end
26
27 endmodule
```

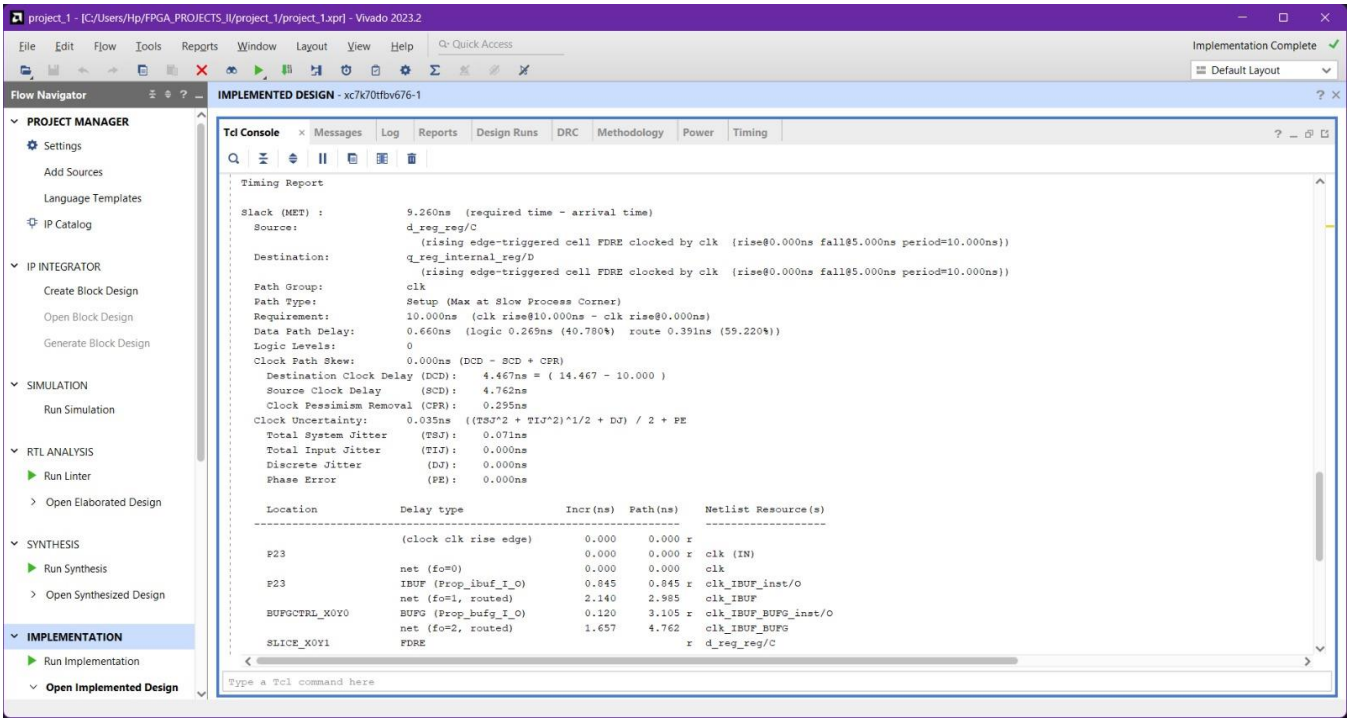
Project Summary | Device | multicycle_demo.v | **multicycle_demo.xdc**

C:/Users/Hp/FPGA_PROJECTS_II/project_1/project_1.srcs/constrs_1/new/multicycle_demo.xdc

```
1 #create_clock -name clk -period 10[get_ports clk]
2 # 100 MHz clock (10 ns period)
3 #create_clock -period 10.000 -name clk [get_ports clk]
4 create_clock -period 10.000 -name clk [get_ports clk]
5 # Relax setup path to 2 clock cycles (20 ns)
6 set_multicycle_path 2 -setup -from [get_cells d_reg_reg] -to [get_cells q_reg_internal_reg]
7 # Adjust hold check back by 1 cycle (to 0 ns)
8 set_multicycle_path 1 -hold  -from [get_cells d_reg_reg] -to [get_cells q_reg_internal_reg]
```







OBSERVATIONS

Before applying the multicycle setup constraint, the path is normally checked against one clock period (10 ns). After applying set_multicycle_path 2 -setup, the selected path is allowed two clock periods (approximately 20 ns) for setup analysis. The exact slack, WNS, TNS, and path delay depend on the synthesized and implemented design and must be copied from the actual Vivado timing report.

RESULT

Multicycle path timing analysis was set up using the set_multicycle_path constraint in Vivado. The selected path's setup requirement is relaxed from one clock cycle to two clock cycles, and the corresponding hold adjustment is included. Final slack values should be recorded from the generated timing report.

CONCLUSION

This experiment demonstrates how a multicycle path exception changes setup timing analysis for a path that is intentionally allowed more than one clock cycle. The experiment also highlights that timing exceptions must be applied only to paths justified by the circuit's functional behaviour.

EXPERIMENT NO. 8

AIM: To study clock-gating timing verification and enable-signal behaviour using Verilog HDL and Xilinx Vivado.

REQUIRED SOFTWARE: Xilinx Vivado Design Suite, Verilog HDL, and Vivado behavioural simulation.

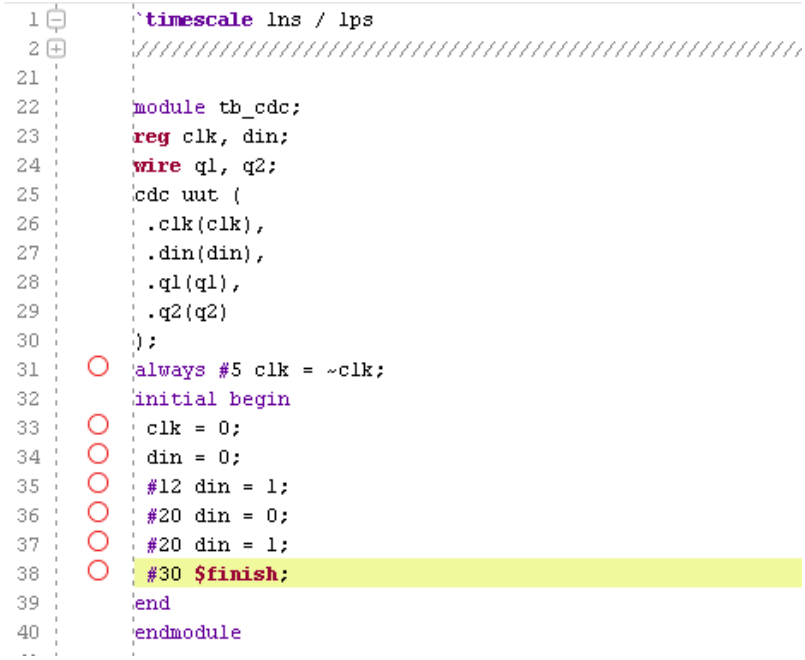
THEORY:

Clock gating is a low-power design technique that reduces dynamic power by preventing unnecessary clock transitions from reaching a circuit when the circuit is not required to operate. Since sequential elements can consume dynamic power as their internal states switch, reducing unnecessary clock activity can reduce power consumption.
An enable signal controls whether a sequential circuit updates its state. When enable is HIGH, a register captures its input at the active clock edge. When enable is LOW, the register retains its previous value. In a practical integrated clock-gating circuit, the enable must be controlled so that it does not create glitches on the gated clock. The supplied Experiment 8 material includes a two-register design named cdc, with no explicit enable input or gated-clock output. Therefore, the supplied code demonstrates two-stage sequential propagation and latency; by itself, it does not implement a clock-gating cell or directly test enable-controlled clock gating. The experiment below preserves that supplied design and identifies this limitation.

DESIGN DESCRIPTION

The design contains two registers, q1 and q2, driven by the same rising-edge clock. q1 captures din at each rising edge, while q2 captures the previous value of q1. Thus, the input passes through two sequential stages. The reset signal is not present in the supplied code.
Block flow: din ⟶ [Register q1] ⟶ [Register q2] ⟶ q2
The testbench uses a 10 ns clock period because the clock toggles every 5 ns.

```
 1        `timescale 1ns / 1ps
 2      + //////////////////////////////////////////////////////////////////
21
22
23
24      - module cdc(
25          input clk,
26          input din,
27          output reg q1,
28          output reg q2
29        );
30      - always @(posedge clk) begin
31          q1 <= din;
32          q2 <= q1;
33      - end
34      - endmodule
35
36
37
```



```
1   `timescale 1ns / 1ps
2   //////////////////////////////////////////////////////////////////
21
22  module tb_cdc;
23  reg clk, din;
24  wire q1, q2;
25  cdc uut (
26   .clk(clk),
27   .din(din),
28   .q1(q1),
29   .q2(q2)
30  );
31  always #5 clk = ~clk;
32  initial begin
33   clk = 0;
34   din = 0;
35   #12 din = 1;
36   #20 din = 0;
37   #20 din = 1;
38   #30 $finish;
39  end
40  endmodule
```

VERILOG CODE (cdc.v)

```
module cdc(
input clk,
input din,
output reg q1,
output reg q2
);
always @(posedge clk) begin
q1 <= din;
q2 <= q1;
end
endmodule
```

TESTBENCH CODE (tb_cdc.v)

```
`timescale 1ns/1ps
module tb_cdc;
reg clk, din;
wire q1, q2;
cdc uut (
.clk(clk),
.din(din),
.q1(q1),
.q2(q2));
always #5 clk = ~clk;
initial begin
clk = 0;
din = 0;
#12 din = 1;
#20 din = 0;
#20 din = 1;
#30 $finish;
```

```
end
endmodule
```

PROCEDURE

1. Open Vivado and create a new RTL project.
2. Add cdc.v under Design Sources.
3. Add tb_cdc.v under Simulation Sources.
4. Set tb_cdc as the simulation top.
5. Select Run Simulation → Run Behavioral Simulation.
6. Display clk, din, q1, and q2 in the waveform window.
7. Observe that q1 samples din at each rising edge and q2 samples the previous value of q1.
8. Check the propagation delay across the two register stages against the input transitions in the testbench.
9. Save the waveform or take a screenshot for the OUTPUT section of the lab record.

WAVEFORM OBSERVATIONS

The clock toggles every 5 ns, giving a 10 ns clock period.
q1 captures the value of din at a rising clock edge.
q2 captures the previous value of q1 at the same rising edge because nonblocking assignments update the registers together.
The exact transition times in the waveform depend on the input transition times and the rising clock edges.
Expected signal sequence: din → q1 (first stage) → q2 (second stage)



RESULT

The supplied two-register cdc design and testbench are prepared for behavioural simulation in Vivado. The expected waveform shows q1 sampling din and q2 following the previous q1 value, producing two-stage latency. The simulation waveform should be attached after running the testbench.

CONCLUSION

This experiment explains the role of sequential stages, enable-controlled activity, and clock gating in low-power and timing-aware digital design. The supplied Verilog code specifically demonstrates two-stage register propagation, not a complete clock-gating implementation. A true clock-gating or enable-signal experiment requires an explicit enable/gating circuit and corresponding waveform verification.

EXPERIMENT-9

Aim: To determine and simulate multi-Mode Multi-Corner (MMMC) Timing Analysis across PVT Corners.

Software Required

- Xilinx Vivado Design Suite
- Verilog HDL
- XDC constraint file
- A PC or laptop

Theory:

Multi-Mode Multi-Corner (MMMC) timing analysis is a Static Timing Analysis (STA) methodology used in VLSI design to verify the timing performance of a digital circuit under different operating modes and Process, Voltage, and Temperature (PVT) conditions.

A digital circuit may operate under different functional conditions, such as normal operation, test mode, or low-power mode. Additionally, variations in semiconductor manufacturing processes, supply voltage, and operating temperature can affect the propagation delay of logic cells and interconnects. Therefore, timing analysis under only one operating condition may not be sufficient to ensure reliable circuit operation.

MMMC analysis evaluates the design under multiple combinations of operating modes and PVT corners to ensure that setup and hold timing requirements are satisfied under the specified conditions.

Multi-Mode Analysis:

Multi-mode analysis verifies the timing behavior of a design under different functional and test configurations. Each mode may have different clock definitions, timing constraints, and active paths.

Common operating modes include:

- Functional mode: The circuit performs its normal intended operation.
- Test mode: The circuit operates under test-related conditions, such as scan testing.
- Low-power mode: The design operates under specified low-power configurations, where applicable.

Each mode may require different timing constraints because clock frequencies, active paths, and input/output requirements can vary.

Multi-Corner Analysis:

Multi-corner analysis evaluates circuit timing under different PVT conditions. PVT stands for Process, Voltage, and Temperature.

The three parameters are explained below.

a) Process corner

Process corners represent manufacturing variations that affect transistor characteristics and cell delays.

- FF (Fast-Fast): Represents a fast-process condition for the modeled NMOS and PMOS devices.
- SS (Slow-Slow): Represents a slow-process condition.
- TT (Typical-Typical): Represents typical modeled process behavior.

b) Voltage corner

Supply voltage affects transistor switching speed. Lower supply voltage generally increases cell delay, while higher supply voltage generally reduces cell delay within the characterized operating range.

c) Temperature corner

Temperature affects transistor and interconnect behavior. Its impact on delay depends on the technology

and operating conditions; higher temperature does not always produce the worst delay.

Important timing terms

- Data arrival time: The time at which data reaches the destination point.
- Data required time: The latest permissible arrival time for data to satisfy the timing requirement.
- Slack: The difference between the required time and arrival time, based on the applicable timing check.
- Critical path: The path with the most restrictive timing result, typically the path with the worst setup slack for setup analysis.
- Clock period: The time interval between consecutive active clock edges.

For setup timing analysis, positive slack indicates that the path meets its timing requirement, while negative slack indicates a timing violation.

Verilog Program

The following program implements a simple 4-bit register-to-register data path with combinational logic between two registers.

```
module delay_analysis (
   input clk,
   input rst,
   input [3:0] a,
   output reg [3:0] y);
reg [3:0] data_reg;
wire [3:0] logic_out;
assign logic_out = (data_reg & 4'b1010) |
                   (data_reg ^ 4'b0101);
always @(posedge clk) begin
   if (rst) begin
      data_reg <= 4'b0000;
      y <= 4'b0000;
   end
   else begin
      data_reg <= a;
      y <= logic_out;
   end
end
endmodule
```

Timing Constraint File (XDC)

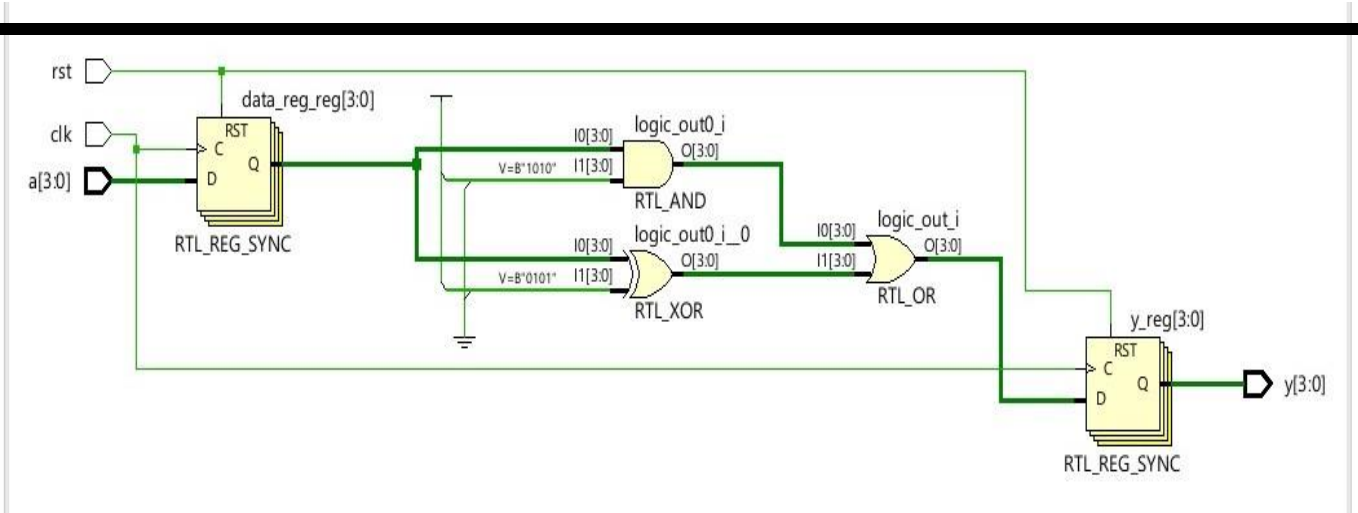
Create a constraint file named delay_analysis.xdc and add the following command:

```
create_clock -name clk -period 10.000 [get_ports clk]
```

Explanation:

- create_clock defines the clock used for timing analysis.
- -name clk specifies the clock name.
- -period 10.000 specifies a clock period of 10 ns, corresponding to 100 MHz.
- [get_ports clk] identifies the top-level clock input port.

This constraint allows Vivado to analyze the register-to-register path against a 10 ns clock period.



Observation and Analysis

After implementation, observe the timing report generated by Vivado. The following parameters are important.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	9.055 ns	Worst Hold Slack (WHS):	0.140 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	4	Total Number of Endpoints:	4	Total Number of Endpoints:	9

All user specified timing constraints are met.

```
----------------------------------------------------------------------------------------------------
From Clock:  clk
  To Clock:  clk

Setup :            0  Failing Endpoints,  Worst Slack        9.055ns,  Total Violation        0.000ns
Hold  :            0  Failing Endpoints,  Worst Slack        0.140ns,  Total Violation        0.000ns
PW    :            0  Failing Endpoints,  Worst Slack        4.500ns,  Total Violation        0.000ns
----------------------------------------------------------------------------------------------------
 Source:                 data_reg_reg[1]/C
                           (rising edge-triggered cell FDRE clocked by clk  {rise@0.000ns fall@5.000ns period=10.000ns})
 Destination:            y_reg[1]/D
                           (rising edge-triggered cell FDRE clocked by clk  {rise@0.000ns fall@5.000ns period=10.000ns})
 Path Group:             clk
 Path Type:              Setup (Max at Slow Process Corner)
 Requirement:            10.000ns  (clk rise@10.000ns - clk rise@0.000ns)
 Data Path Delay:        0.841ns  (logic 0.456ns (54.220%)  route 0.385ns (45.780%))
 Logic Levels:           0
 Clock Path Skew:        -0.022ns (DCD - SCD + CPR)
   Destination Clock Delay (DCD):    4.346ns = ( 14.346 - 10.000 )
   Source Clock Delay      (SCD):    4.707ns
   Clock Pessimism Removal (CPR):    0.339ns
 Clock Uncertainty:      0.035ns  ((TSJ^2 + TIJ^2)^1/2 + DJ) / 2 + PE
   Total System Jitter     (TSJ):    0.071ns
   Total Input Jitter      (TIJ):    0.000ns
   Discrete Jitter          (DJ):    0.000ns
   Phase Error              (PE):    0.000ns

   Location             Delay type                Incr(ns)  Path(ns)    Netlist Resource(s)
-------------------------------------------------------------------    -------------------
                        (clock clk rise edge)        0.000     0.000 r
   N13                                               0.000     0.000 r  clk (IN)
                        net (fo=0)                   0.000     0.000    clk
   N13                  IBUF (Prop_ibuf_I_O)         1.001     1.001 r  clk_IBUF_inst/O
                        net (fo=1, routed)           1.972     2.973    clk_IBUF
```

```
                     net (fo=1, routed)           1.972     2.973    clk_IBUF
  BUFGCTRL_X0Y0      BUFG (Prop_bufg_I_O)         0.096     3.069 r  clk_IBUF_BUFG_inst/O
                     net (fo=8, routed)           1.638     4.707    clk_IBUF_BUFG
  SLICE_X1Y5         FDRE                                       r  data_reg_reg[1]/C
-----------------------------------------------------------------    -------------------
  SLICE_X1Y5         FDRE (Prop_fdre_C_Q)         0.456     5.163 r  data_reg_reg[1]/Q
                     net (fo=1, routed)           0.385     5.548    data_reg[1]
  SLICE_X0Y5         FDRE                                       r  y_reg[1]/D
-----------------------------------------------------------------    -------------------

                     (clock clk rise edge)       10.000    10.000 r
  N13                                            0.000    10.000 r  clk (IN)
                     net (fo=0)                  0.000    10.000    clk
  N13                                            0.000    10.000 r  clk (IN)
                     net (fo=0)                  0.000    10.000    clk
  N13                IBUF (Prop_ibuf_I_O)        0.867    10.867 r  clk_IBUF_inst/O
                     net (fo=1, routed)          1.868    12.735    clk_IBUF
  BUFGCTRL_X0Y0      BUFG (Prop_bufg_I_O)        0.091    12.826 r  clk_IBUF_BUFG_inst/O
                     net (fo=8, routed)          1.519    14.346    clk_IBUF_BUFG
  SLICE_X0Y5         FDRE                                       r  y_reg[1]/C
                     clock pessimism             0.339    14.685
                     clock uncertainty          -0.035    14.650
  SLICE_X0Y5         FDRE (Setup_fdre_C_D)      -0.047    14.603    y_reg[1]
-----------------------------------------------------------------
                     required time                        14.603
                     arrival time                         -5.548
-----------------------------------------------------------------
                     slack                                 9.055
```

Parameter	Description
Data Path Delay	Delay through the data path, including cell and routing delays
Data Arrival Time	Time at which data reaches the destination register
Data Required Time	Latest permissible arrival time for the timing check
Setup Slack	Difference between the required time and arrival time for setup analysis
Logic Delay	Delay contributed by logic elements
Routing Delay	Delay contributed by signal interconnections
Clock Period	Defined clock period of 10 ns

Sample observation table

S. No.	Parameter	Observed value
1	Clock period	10 ns
2	Clock frequency	100 MHz
3	Data path delay	0.841 ns
4	Logic delay	0.456 ns
5	Routing delay	0.385 ns
6	Worst setup slack	9.055 ns
7	Timing status	Pass

Result: The total path delay of the designed circuit was analyzed using Vivado Static Timing Analysis. The timing report was used to examine data arrival time, logic delay, routing delay, and setup slack, and to identify the critical timing path

EXPERIMENT-10

Aim: To determine On-Chip Variation (OCV/AOCV/SOCV) and Timing Derating Analysis.

Software Required

- Xilinx Vivado Design Suite
- Verilog HDL
- XDC constraint file

Theory:

On-Chip Variation (OCV) is a timing analysis technique used in Static Timing Analysis (STA) to account for variations in the delay of cells and interconnects across a single integrated circuit. These variations can occur due to manufacturing process differences, voltage fluctuations, temperature changes, and physical differences across the chip.

In VLSI design, different cells on the same chip may exhibit slightly different delays even when they are designed to perform the same function. If these variations are not considered, the circuit may experience setup or hold timing violations. OCV-based analysis provides a more conservative and realistic estimation of timing behavior.

1. On-Chip Variation (OCV)

OCV models delay variations by applying fixed timing derating factors to cell and interconnect delays. A derating factor increases or decreases the nominal delay to represent possible variations.

- Setup analysis: Data-path delays are generally increased, while relevant capture clock-path delays may be reduced to check the worst-case setup condition.
- Hold analysis: Data-path delays are generally reduced, while relevant capture clock-path delays may be increased to check the worst-case hold condition.

2. Advanced On-Chip Variation (AOCV)

- AOCV is an improved variation analysis method that considers factors such as logic depth and physical distance when determining timing derating.
- Unlike conventional OCV, which often applies fixed derating factors, AOCV uses characterized tables to determine the appropriate variation based on the number of cells in a path and, depending on the methodology, the physical distance or path characteristics.
- AOCV provides more accurate timing analysis and can reduce excessive pessimism while maintaining adequate timing margins.

Important commands

Command	Purpose
create_clock	Defines a clock on a physical clock port
create_clock without a port target	Defines a virtual clock
set_input_delay	Specifies external input timing
set_output_delay	Specifies external output timing
report_timing	Displays detailed timing paths
report_timing_summary	Displays the overall timing status

Verilog Program

The following circuit is a simple 4-bit registered data-processing circuit.

```
module virtual_clock_analysis (
    input clk,
    input rst,
```

```
input [3:0] data_in,
   output reg [3:0] data_out);
always @(posedge clk) begin
   if (rst)
      data_out <= 4'b0000;
   else
      data_out <= data_in;
end
endmodule
```

```
`timescale 1ns / 1ps
//////////////////////////////////////////////////////////////////////////////////


module virtual_clock_analysis (
    input clk,
    input rst,
    input [3:0] data_in,


    output reg [3:0] data_out
);
always @(posedge clk) begin
    if (rst)
        data_out <= 4'b0000;
    else
        data_out <= data_in;
end
endmodule
```

Circuit operation:

- data_in is a 4-bit input.
- clk controls when the input data is sampled.
- rst clears the output register synchronously.
- data_out stores the sampled input data.

This circuit allows analysis of input-to-register and register-to-output timing paths. Because it has no combinational logic between the input and register, the measured delays may be small.

5. XDC Timing Constraint File

Create a constraint file named virtual_clock_analysis.xdc and add the following constraints.

```
create_clock -name sys_clk -period 10.000 [get_ports clk]

# Virtual clock for external interface timing
create_clock -name virt_clk -period 10.000

# Input delay constraints
set_input_delay -clock virt_clk -max 2.000 [get_ports {data_in[*]}]
set_input_delay -clock virt_clk -min 0.500 [get_ports {data_in[*]}]

# Output delay constraints
set_output_delay -clock virt_clk -max 2.000 [get_ports {data_out[*]}]
set_output_delay -clock virt_clk -min 0.500 [get_ports {data_out[*]}]
```

```
# Actual clock connected to the FPGA
create_clock -name sys_clk -period 10.000 [get_ports clk]

# Virtual clock for external interface timing
create_clock -name virt_clk -period 10.000

# Input delay constraints
set_input_delay -clock virt_clk -max 2.000 [get_ports {data_in[*]}]
set_input_delay -clock virt_clk -min 0.500 [get_ports {data_in[*]}]

# Output delay constraints
set_output_delay -clock virt_clk -max 2.000 [get_ports {data_out[*]}]
set_output_delay -clock virt_clk -min 0.500 [get_ports {data_out[*]}]

```

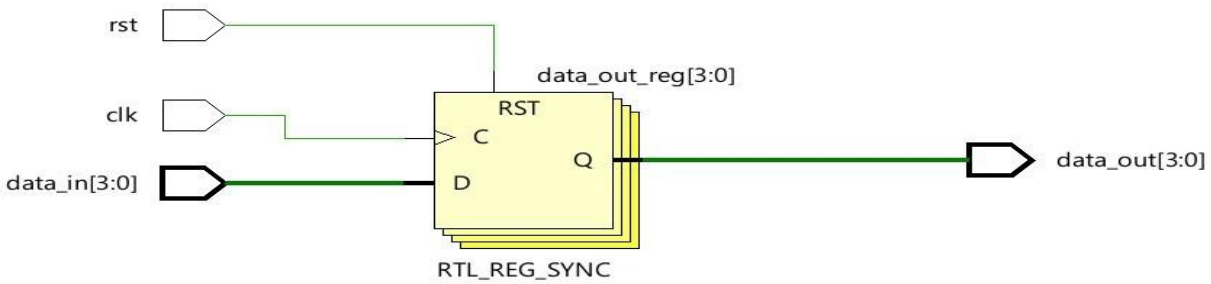
Explanation

- sys_clk defines a 10 ns clock on the actual clock port.
- virt_clk defines a virtual clock with a 10 ns period.
- set_input_delay -max 2.000 specifies a maximum external input delay of 2 ns.
- set_input_delay -min 0.500 specifies a minimum external input delay of 0.5 ns.
- set_output_delay -max 2.000 specifies a maximum output timing requirement of 2 ns relative to the virtual clock.
- set_output_delay -min 0.500 specifies the minimum output delay value used for the relevant timing checks.

Important: These are example constraint values for the experiment, not measured device specifications. The actual values should ideally come from the external device's timing specifications.

Procedure to Perform in Vivado

1. Open Vivado and create a new RTL project.
2. Add virtual_clock_analysis.v as a design source.
3. Add virtual_clock_analysis.xdc as a constraint file.
4. Select the correct device or board.
5. Run Synthesis and check for errors.
6. Run Implementation.
7. Open the implemented design.
8. Select Reports → Timing → Report Timing Summary.
9. Examine the input-to-register and register-to-output timing paths.
10. Review the reported arrival times, required times, and slack.



Observation Table

Record the actual values from your Vivado reports.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	-1.381 ns	Worst Hold Slack (WHS):	0.176 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	-5.515 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	4	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	8	Total Number of Endpoints:	8	Total Number of Endpoints:	5

Timing constraints are not met.

```
| Clock Summary
| -------------

Clock     Waveform(ns)       Period(ns)      Frequency(MHz)
-----     ------------       ----------      --------------
sys_clk   {0.000 5.000}      10.000          100.000
virt_clk  {0.000 5.000}      10.000          100.000


From Clock:  sys_clk
  To Clock:  virt_clk

Setup :            4  Failing Endpoints,  Worst Slack       -1.381ns,  Total Violation       -5.515ns
Hold  :            0  Failing Endpoints,  Worst Slack        3.559ns,  Total Violation        0.000ns


From Clock:  virt_clk
  To Clock:  sys_clk

Setup :            0  Failing Endpoints,  Worst Slack        5.704ns,  Total Violation        0.000ns
Hold  :            0  Failing Endpoints,  Worst Slack        0.176ns,  Total Violation        0.000ns
```

S. No.	Parameter	Observed value
1	System clock period	10 ns
2	Virtual clock period	10 ns
3	Maximum input delay	2 ns
4	Minimum input delay	0.5 ns
5	Maximum output delay	2 ns
6	Minimum output delay	0.5 ns
7	Input-to-register slack	5.704 ns (setup), 0.176 ns (hold)
8	Register-to-output slack	-1.381 ns (setup, VIOLATED), 3.559 ns (hold)

Result:

Virtual clock, input delay, and output delay constraints were defined in Vivado using XDC commands. The timing reports were examined to understand their effect on input-to-register and register-to-output timing analysis.

EXPERIMENT-11

Aim: To analyse Clock Domain Crossing (CDC) Analysis using Synchronizer Circuits.

Software Required

- Xilinx Vivado Design Suite
- Verilog HDL
- XDC constraint file
- PC or laptop

Theory:

Clock Domain Crossing (CDC) is a technique used to transfer signals between two different clock domains in a digital circuit. A clock domain is a part of a design controlled by a particular clock signal. When data is transferred between circuits operating with different clock frequencies or unrelated clock phases, CDC problems may occur.

A CDC occurs when a signal generated in one clock domain is sampled by a register operating in another clock domain.

For example, consider a design containing two clock domains:

- Clock Domain 1: Operates at 100 MHz.
- Clock Domain 2: Operates at 50 MHz.

When a signal generated using the first clock is sampled using the second clock, the signal transition may occur close to the sampling edge of the destination clock. This can violate the setup or hold requirements of the destination register.

CDC problems commonly occur in systems containing multiple clock frequencies, independent clock sources, asynchronous external inputs, or separate functional blocks.

Metastability is a condition in which a flip-flop temporarily enters an unstable state because its input changes close to the active clock edge, violating setup or hold timing requirements.

During metastability, the output may take longer than expected to settle to a valid logic level of 0 or 1. If the unstable output is used directly by other logic, it may cause incorrect operation.

Metastability cannot be completely eliminated in practical synchronous digital circuits, but its probability of propagating can be reduced significantly through proper synchronizer design.

Verilog Program

The following circuit contains two registers with combinational logic between them. It can be used to study register-to-register timing in Vivado.

```
module setup_hold_analysis (
   input clk,
   input rst,
   input a,
   output reg y
);
reg q1;
wire n1, n2, n3;
assign n1 = q1 ^ a;
assign n2 = n1 & a;
assign n3 = n2 | q1;
always @(posedge clk) begin
   if (rst) begin
      q1 <= 1'b0;
      y  <= 1'b0;
```

```
  end
   else begin
      q1 <= a;
      y  <= n3;
   end
end
endmodule
```

```
21  module setup_hold_analysis(
22      input clk,
23      input rst,
24      input a,
25      output reg y);
26  reg q1;
27  wire n1, n2, n3;
28  assign n1 = q1 ^ a;
29  assign n2 = n1 & a;
30  assign n3 = n2 | q1;
31  always @(posedge clk) begin
32      if (rst) begin
33          q1 <= 1'b0;
34          y  <= 1'b0;
35      end
36      else begin
37          q1 <= a;
38          y  <= n3;
39      end
40  end
41  endmodule
```

Explanation:

- q1 is the first register.
- n1, n2, and n3 form combinational logic.
- y is the destination register.
- Both registers operate on the rising edge of clk.
- Vivado can analyze the timing path between registers and report its delay and slack.

Note that synthesis may simplify the Boolean logic. The final implemented circuit, rather than the original RTL alone, determines the timing results.

Timing Constraint File (XDC)

Create a file named setup_hold_analysis.xdc and enter:

create_clock -name clk -period 10.000 [get_ports clk]

This constraint defines a clock period of 10 ns, corresponding to 100 MHz.

f={1}/{T}={1}/{10{ ns}}=100{ MHz}]

Vivado uses this clock constraint to analyze register-to-register setup timing. Hold timing is also evaluated using the applicable clock and device timing models.

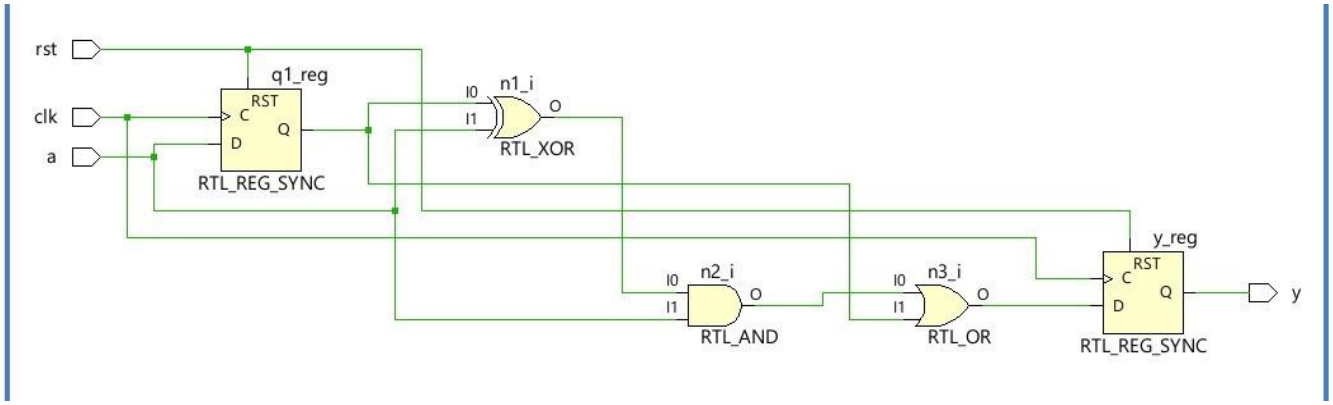


```
1  create_clock -name clk -period 10.000 [get_ports clk]
```

Procedure

1. Open Vivado and create a new RTL project.
2. Add the Verilog source file setup_hold_analysis.v.
3. Add the XDC file containing the clock constraint.
4. Select the target device or board.
5. Run Run Synthesis and check for errors.
6. Run Run Implementation.
7. Open the implemented design.

8. Select Reports → Timing → Report Timing Summary.
9. Examine the setup and hold results, including WNS and TNS.
10. Generate detailed timing reports to inspect the startpoint, endpoint, data-path delay, and slack.
11. If violations exist, review the timing paths and apply appropriate optimization.
12. Run implementation again and compare the timing reports before and after optimization.



Observation Table

Record the actual values from your Vivado reports.

Design Timing Summary

Setup		Hold		Pulse Width	
Worst Negative Slack (WNS):	9.111 ns	Worst Hold Slack (WHS):	0.198 ns	Worst Pulse Width Slack (WPWS):	4.500 ns
Total Negative Slack (TNS):	0.000 ns	Total Hold Slack (THS):	0.000 ns	Total Pulse Width Negative Slack (TPWS):	0.000 ns
Number of Failing Endpoints:	0	Number of Failing Endpoints:	0	Number of Failing Endpoints:	0
Total Number of Endpoints:	1	Total Number of Endpoints:	1	Total Number of Endpoints:	3

All user specified timing constraints are met.

```
From Clock:  clk
  To Clock:  clk

Setup :            0  Failing Endpoints,  Worst Slack        9.111ns,  Total Violation        0.000ns
Hold  :            0  Failing Endpoints,  Worst Slack        0.198ns,  Total Violation        0.000ns
PW    :            0  Failing Endpoints,  Worst Slack        4.500ns,  Total Violation        0.000ns
```

Parameter	Before optimization	After optimization
Clock period	10 ns	10 ns
Setup WNS	9.111 ns	N/A (timing already met)
Setup TNS	0.000 ns	N/A (timing already met)
Hold WNS	0.198 ns	N/A (timing already met)
Hold TNS	0.000 ns	N/A (timing already met)
Timing status	All constraints met (no violations)	N/A (no optimization required)

The values must be taken from the actual reports. This example does not guarantee that the design will produce setup or hold violations.

Optimization and Analysis

Vivado performs timing-driven optimization during synthesis and implementation. It may restructure logic, select different resources, adjust placement, and modify routing to improve timing.

For setup violations, optimization aims to reduce the maximum data-path delay or improve the applicable clock relationship.
For hold violations, implementation may insert delay elements or buffers to increase minimum data-path delay. These changes must be checked against setup timing because fixing hold can adversely affect maximum-delay paths.
Important: A small design may meet both setup and hold requirements without any violations. To demonstrate optimization meaningfully, use a design and constraints that produce a measurable timing issue, then compare the reports. Do not introduce arbitrary delay logic and assume that it will create a specific violation.

9. Result:

Setup and hold timing analysis was performed using Vivado Static Timing Analysis. The timing reports were used to examine path delays, slack, and timing status, and to study the effect of implementation optimization on timing performance.